LEUPHANA
UNIVERSITÄT LÜNEBURG

Projektbericht

KI-unterstützte Produktentwicklung

Spätestes Abgabedatum: 31.03.2026

Automatisierte Anforderungs-extraktion aus Pflichtenheften: Ein Vergleich zwischen einer RAG-basierten und zweistufigen LLM-Methode

Autoren:

Name	Matrikelnummer
Jan Ole Trosin	4006102
Luca Russek	4006090
Pia Voigtländer	3048228
Mika Wettstaedt	3047261
Irina Kireeva	3047015
Raphael Wohlers	3047464

Projektgruppe: Gruppe 2 - Pflichtenheftanalyse

Dozent: Prof. Dr. Arthur Seibel

Inhaltsverzeichnis

Eidesstattliche Erklärung III
Hinweis zur Sprache IV
Aufgabenstellung – Pflichtenheftanalyse V
Abbildungsverzeichnis VI
Tabellenverzeichnis VI
Abkürzungsverzeichnis VII

1 Einleitung (Pia Voigtländer) 1
2 Stand der Technik 2
2.1 Qualitätsmerkmale guter Anforderungen nach ISO/IEC/IEEE 29148 (Raphael Wohlers) 2
2.2 Chunking (Pia Voigtländer) 3
2.3 Embedding (Pia Voigtländer) 5
2.4 RAG-System (Mika Wettstaedt) 6
2.5 API-Schnittstelle (Mika Wettstaedt) 8
3 Zielsetzung (Pia Voigtländer) 10
4 Methodik 11
4.1 Voreinstellung der beiden Programmieransätze (Jan Ole Trosin) 11
4.2 Erster Ansatz: Doppelstufige LLM-Methode (Jan Ole Trosin) 17
4.3 Zweiter Ansatz: Eigenes RAG-System (Irina Kireeva) 25
4.4 Bewertungsschema (Raphael Wohlers) 31
5 Ergebnisse 34
5.1 Bewertung der Ergebnisse – Erster Ansatz (Luca Russek) 34
5.2 Bewertung der Ergebnisse – Zweiter Ansatz (Lucas Russek) 36
6 Diskussion & Einordnung (Luca Russek) 38
7 Fazit & Ausblick (Luca Russek) 41

Quellenverzeichnis VIII
Anhangsverzeichnis XII

Eidesstattliche Erklärung

Hiermit erklären wir an Eides statt, dass wir die vorliegende Arbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt haben. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form weder von uns noch von jemand anderem als Prüfungsleistung vorgelegt.

Name	Ort & Datum	Unterschrift
Jan Ole Trosin	Lüneburg, 29.03.2026	
Mika Wettstaedt	Lüneburg, 29.03.2026	
Raphael Wohlers	Lüneburg, 29.03.2026	
Luca Russek	Lüneburg, 29.03.2026	
Irina Kireeva	Lüneburg, 29.03.2026	
Pia Voigtländer	Lüneburg, 29.03.2026	

Hinweis zur Sprache

Aus dem Grund der besseren Lesbarkeit wird in der gesamten angefertigten Projektarbeit im Modul „KI-unterstütze Produktentwicklung“ das generische Maskulinum verwendet. Durch die Voreinstellung der verwendeten formalen Schriftsprache werden somit alle Personenbezeichnungen aller Geschlechter mit einbezogen, außer wenn es in anderen Formulierungen explizit kenntlich gemacht wird, dass ein anderes Geschlecht angesprochen wird.

Aufgabenstellung – Pflichtenheftanalyse

Sie sind ein Systemintegrator für Lagerlogistik und haben für den Bau eines automatisierten Schmalganghochregallagers ein Angebot verfasst, welches angenommen wurde. Um dieses Angebot umzusetzen, benötigt ihre Produktentwicklung eine ISO-konforme Anforderungsliste. Erstellen Sie eine LLM-basierte Lösung, die aus dem Angebot eine solche Liste erstellt. Dabei soll jeder Anforderung eine der vier Standardmethoden zur Validierung (Inspektion, Analyse, Demonstration, Test) zugeordnet und ein konkreter Umsetzungsvorschlag gemacht werden.

Abbildungsverzeichnis

Abbildung 1: Parallele Bearbeitung aller Datentypen 12
Abbildung 2: Unterteilte Verarbeitungsstufen der doppelstufigen LLM-Methode 18
Abbildung 3: Unterteilte Verarbeitungsstufen des RAG-Ansatzes 26

Tabellenverzeichnis

Tabelle 1: Liste der extern importierten Bibliotheken für die dopplestufigen LLM Methode ... 19
Tabelle 2: Liste der RAG-spezifischen, extern importierten Bibliotheken 27

Abkürzungsverzeichnis

API	Application Programming Interface
LLM	Large Language Modell
NLP	Natural Language Processing
RAG	Retrieval Augmented Generation
SAIA	Scalable AI Accelerator

1 Einleitung (Pia Voigtländer)

Laut dem Modell der Produktentwicklung (VDI 2221-1) ist der erste Schritt nach der Problemfeststellung bzw. der Aufgabendefinierung, das Präzisieren des Problems bzw. der Aufgabe. Dafür sollte idealerweise ein Austausch mit allen Stakeholdern stattfinden. Das Ergebnis dieses Prozessschrittes ist eine Liste von Anforderungen. Insbesondere wenn eine klare Trennung zwischen Auftraggeber und Auftragnehmer vorhanden ist, stellt häufig ein Angebot die Basis für die Anforderungsliste dar. Das genaue Verständnis des Problems, welches dem Angebot zugrunde liegt und die Umwandlung des Angebots zu einer Anforderungsliste ist ein wichtiger Schritt in der Produktentwicklung. Vor allem zu Beginn eines Projektes ist die Möglichkeit der Kostenbeeinflussung sehr hoch und nimmt mit fortlaufendem Projektfortschritt stark ab (vgl. Ehrlenspiel, et al., 2020). Zudem sind die Kosten, welche Fehler auslösen, in diesem Projektabschnitt noch gering, während sie später exponentiell ansteigen können (vgl. Lindemann, 2009). Vor diesem Hintergrund sollte dieser Prozess möglichst gewissenhaft durchgeführt werden. Da es sich dabei um eine sehr zeitintensive Aufgabe handeln kann, wurde bereits damit begonnen zu untersuchen wie LLMs (Large-Language-Modell) bei dieser Aufgabe eingesetzt werden können (siehe beispielsweise vgl. Schiller, et al., 2025).

In der vorliegenden Arbeit werden zwei unterschiedliche LLM-Ansätze bewertet und verglichen, welche die Erstellung der Anforderungsliste automatisieren sollen. Bei dem ersten Ansatz handelt es sich um eine doppelstufige LLM-Methode, welches eine lineare Pipeline darstellt. Hierbei wird das Angebot zunächst in kleine Abschnitte (auch Chunks genannt) aufgeteilt. Aus jedem einzelnen Abschnitt versucht ein LLM Anforderungen für die Anforderungsliste zu generieren. Ein zweites LLM kontrolliert die erstellte Anforderungsliste und führt die Prüfung nach den INCOSE-Regeln durch. Den zweiten Ansatz stellt ein RAG-System dar. Auch hier wird damit begonnen das Angebot in Chunks aufzuteilen. Diese werden allerdings einzeln in Vektoren umgewandelt und in einer Vektordatenbank gespeichert. Die Anforderungen für die Anforderungsliste werden erzeugt indem gezielte Abfragen an die Vektordatenbank gestellt werden. Auch bei diesem Ansatz werden die erzeugten Anforderungen durch ein zweites LLM auf die Einhaltung der INCOSE-Regeln überprüft.

Das Ziel dieser Arbeit ist es herauszufinden, wie sich die Unterschiede der beiden Ansätze in der methodischen Architektur auf die Extraktion der Anforderungen in der Produktentwicklung auswirken. Dafür werden diese beide Ansätze ausgearbeitet und ihre Ergebnisse jeweils bewertet und verglichen. Zudem soll untersucht werden, inwieweit die lückenlose Rückverfolgung der generierten Anforderungen auf ihre ursprüngliche Textstelle im Angebot bei den beiden Ansätzen möglich ist.

2 Stand der Technik

In diesem Kapitel werden die theoretischen und maßgebenden Grundlagen vorgestellt, die für das Verständnis der in Kapitel 4 beschriebenen Programmieransätze erforderlich sind. Zunächst wird mit der ISO/IEC/IEEE 29148 die normative Grundlage des Requirements Engineering erläutert, da sie die Qualitätsanforderungen an gut formulierte Anforderungen definiert und damit den Maßstab für die spätere Bewertung der KI-generierten Anforderungen bildet. Anschließend werden die technischen Grundlagen des Chunkings, des Embeddings, der RAG-Systematik sowie der Nutzung von API-Schnittstellen dargestellt, um die Funktionsweise der implementierten Ansätze nachvollziehbar zu machen.

2.1 Qualitätsmerkmale guter Anforderungen nach ISO/IEC/IEEE 29148 (Raphael Wohlers)

Damit die Qualität der erzeugten Anforderungen in dieser Arbeit nicht nur technisch, sondern auch fachlich bewertet werden kann, ist zunächst eine reglementierte Grundlage des Requirement-Engineerings erforderlich. Eine zentrale Referenz stellt hierbei die ISO/IEC-/IEEE 29148 dar, die Prozesse, Informationsprodukte und Qualitätsmerkmale des Requirement-Engineerings für Systeme und Software beschreibt. Die Norm spezifiziert in Kapitel 5.2.5 & 5.2.6 Bedarfe an die Erstellung und Dokumentation von Anforderungen und definiert damit einen anerkannten Bezugsrahmen für die Beurteilung dieser (vgl. International Organization for Standardization, 2018).

Für den Kontext dieser Arbeit ist insbesondere relevant, dass Anforderungen nicht allein inhaltlich vorhanden sein müssen, sondern auch bestimmten Qualitätsansprüchen genügen sollen. Dazu zählt insbesondere eine klare und eindeutige Formulierung, ausreichende Vollständigkeit, Rückverfolgbarkeit auf ihre Quelle sowie die prinzipielle Prüfbarkeit/Validierbarkeit (vgl. International Organization for Standardization, 2018). Darüber hinaus wird in der Requirements-Engineering Literatur im Umfeld der Norm gefordert, dass Anforderungen möglichst konsistent, nicht redundant und in sich abgrenzbar formuliert werden (vgl. Zowghi & Gervasi, 2002). Diese Merkmale sind für die vorliegende Untersuchung von zentraler Bedeutung, da sie als Grundlage für die Bewertung der durch die beiden KI-Ansätze generierten Anforderung dienen.

Nachdem mit der ISO/IEC-/IEEE 29148 die fachliche Grundlage für die Bewertung guter Anforderungen erläutert wurde, wird als nächster Schritt die Frage beantwortet, wie ein Pflichtenheft technisch so aufbereitet werden kann, dass daraus automatisierte Anforderungen generiert werden können.

2.2 Chunking (Pia Voigtländer)

Sowohl bei dem RAG-Ansatz als auch bei der doppelstufigen LLM-Methode ist das Chunking ein essenzieller Schritt für die Vorbereitung des Inputs, in diesem Anwendungsfall ein Pflichtenheft. Chunking beschreibt den Prozess des Zerteilens von Input-Dokumenten in handliche Abschnitte. Für beide betrachteten Ansätze hat das Chunking einen Einfluss auf den Inhalt und die Qualität des zu erzeugenden Ergebnisses. Bei dem RAG-Ansatz werden die erstellten Abschnitte jeweils als ein Vektor in einer Vektordatenbank eingespeichert und von dort abgerufen. Für die doppelstufige LLM-Methode sind die Chunks direkte Grundlage für die Generierung von Anforderungen. Die größte Herausforderung beim Chunking ist es Abschnitte zu erschaffen, welche groß genug sind, um aussagekräftige Informationen zu erhalten und gleichzeitig klein genug, um möglichst spezifisch zu sein und einen reibungslosen Ablauf der Anwendung sicherstellen zu können (vgl. Schwaber-Cohen & Patel, 2025). Generell können die Chunking-Methoden in fünf Ansätze eingeteilt werden. In tatsächlichen Anwendungen kann es jedoch auch zu Kombinationen von diesen Ansätzen kommen.

Den einfachsten Ansatz stellt das Fixed-Size Chunking dar. Hierbei wird die Anzahl von Tokens pro Chunk festgelegt. Das Input-Dokument wird in Chunks der festgelegten Größe aufgeteilt. Dieses Vorgehen ist einfach zu implementieren und benötigt verhältnismäßig wenig Rechenkapazität. Allerdings werden semantische als auch syntaktische Strukturen missachtet, was häufig zu abgeschnittenen Sätzen und Wörtern führen kann (vgl. Amiri & Bocklitz, 2025). Darunter kann die inhaltliche Qualität und das Verständnis der Chunks leiden. Um diesem Problem zumindest teilweise entgegenzuwirken, können optional Überlappungen zwischen den Chunks integriert werden.

Der Recursive-Token-Chunker (vgl. Chase, 2026) ist ein Chunking-Ansatz, welcher die Struktur des Texts berücksichtigt. Dabei wird zunächst eine Reihenfolge an Satz- und Leerzeichen festgelegt, nach welcher das Input-Dokument aufgeteilt werden soll. Diese Aufteilung erfolgt so lang bis die Textabschnitte die vorgegebene Chunk-Größe erreichen. Dadurch wird ein einfaches syntaktisches Verständnis in das Chunking integriert und die Anzahl von semantisch inkohärenten Chunks reduziert (vgl. Amiri & Bocklitz, 2025).

Das semantische Chunking, zuerst entwickelt von Greg Kamradt, geht noch detaillierter auf die inhaltliche Ebene des Input-Dokumentes ein. Zunächst findet eine Aufteilung wie bei dem Recursive-Token-Chunker anhand von Satz- und Leerzeichen statt. Diese Abschnitte werden dann mit festgelegten Überlappungen in eine Vektordatenbank eingebettet. Die endgültige Chunkgrenze wird zwischen zwei benachbarten Vektoren gesetzt, wenn diese eine festgelegte Kosinus-Ähnlichkeit nicht mehr erfüllen und von einer semantischen Diskontinuität ausgegangen werden kann (vgl. Amiri & Bocklitz, 2025). Durch diesen Ansatz werden Chunks erzeugt,

welche eine thematische Einheit darstellen. In praktischen Anwendungen können die verschiedenen Längen der Chunks zu Problemen führen, wenn die Token-Begrenzung des jeweiligen Modells überschritten wird. Eine überarbeitete Version dieses Ansatzes beinhaltet zusätzlich einen binären Suchalgorithmus, welcher die Position der Chunkgrenze dynamisch anpasst. Somit wird sichergestellt, dass eine gewisse Chunklänge nicht überschritten wird.

Bei dem Cluster-Semantic-Chunking wird der Fokus auf den globalen inhaltlichen Zusammenhang gelegt. Dafür wird das Input-Dokument zunächst in kleine Segmente aufgeteilt. Diese werden jeweils in eine Vektordatenbank eingebettet. In der Vektordatenbank bildet ein dynamischer Algorithmus Gruppen, sodass die semantische Ähnlichkeit individuell maximiert werden kann. Aus diesen Gruppen werden die Chunks gebildet, welche hohe Kohärenzen aufweisen und breitere thematische Kontinuitäten bewahren. Nachteilig an diesem Ansatz ist die große benötigte Rechenkapazität (vgl. Amiri & Bocklitz, 2025).

Den letzten Ansatz stellt der LLM-Semantic-Chunker dar. Dafür wird das Input-Dokument in kleine Abschnitte aufgeteilt und direkt an ein LLM gegeben. Diesem wird in einem Prompt erklärt, dass es semantisch passende Chunkgrenzen identifizieren soll. Somit kann das tiefe Verständnis von Textstrukturen und inhaltlichem Kontext genutzt werden, welches LLMs in der Regel besitzen. Dieser Ansatz hebt sich damit von den anderen ab, da er generell den Kontext betrachtet und dies nicht auf statistischen Ähnlichkeitswerten beruht wie bei dem Embedding. So erzeugte Chunks sind oft sehr kohärent und menschenähnlich, benötigen allerdings auch die größte Rechenleistung (vgl. Amiri & Bocklitz, 2025).

Welcher Ansatz sich in der Anwendung am besten eignet hängt von verschiedenen Merkmalen ab. Eines der wichtigsten Merkmale ist die Länge der Input-Dokumente. Sehr kurze Dokumente müssen möglicherweise gar nicht gechunkt werden, während die Struktur von längeren Dokumenten bereits wegweisend für die Entscheidung der Chunking-Strategie sein kann (vgl. Schwaber-Cohen & Patel, 2025). Zudem sollte der folgende Prozessschritt, das Embedding, berücksichtigt werden. Embedding-Modelle können unterschiedlich große Kapazitäten zur Informationsverarbeitung haben und sind teilweise auf spezifische Fachbereiche und/oder Dateiformate spezialisiert (vgl. Schwaber-Cohen & Patel, 2025). Die Chunking-Strategie sollte darauf abgestimmt werden. In Bezug auf RAG-Anwendungen sollte auch die erwartete Länge und Komplexität der Abfragen beachtet werden, da immer das Ziel sein sollte eine möglichst hohe Korrelation zwischen Abfragevektor und Chunkvektoren zu erreichen (vgl. Schwaber-Cohen & Patel, 2025).

2.3 Embedding (Pia Voigtländer)

Eine in der Literatur verbreitete Definition für das Wort „Embedding“ ist, dass es sich dabei um dichte, verteilte Wortvektoren fester Länge handelt, welche gemäß der Verteilungshypothese anhand von Wortstatistiken erstellt werden (vgl. Almeida & Xexéo, 2023). Es handelt sich also um den Prozess, der Wörter oder Texte in Vektoren umwandelt, damit sie maschinell einfacher verglichen werden können. In so gut wie allen Anwendungen, in welchen die Verarbeitung von natürlicher Sprache stattfindet, muss dieser Prozess durchlaufen werden. Es gibt verschiedene Ansätze, wie die Repräsentation von Text umgewandelt in Vektoren erfolgen kann. Diese Ansätze können, je nach Strategie einer von insgesamt zwei Gruppen zugeteilt werden (vgl. Almeida & Xexéo, 2023).

Die erste Gruppe stellt die Count-Based Modelle dar. Diese betrachten jedes Wort einzeln und ermitteln die Wichtigkeit sowie den Bezug zu anderen Wörtern über die Häufigkeit des Auftretens. Dieser Ansatz baut auf dem Gedanken auf, dass Wörter, welche häufig zusammen genannt werden, eine ähnliche Bedeutung haben (vgl. Hoque, 2024). Solche Modelle arbeiten verhältnismäßig schnell und einfach, allerdings sind sie nicht so sensibel, wenn es um subtile Wortinterpretationen geht (vgl. Hoque, 2024).

Die zweite Gruppe stellt die Prediction-Based Modelle dar. Diese greifen bei der Erzeugung eines Vektors auf den Kontext im Text zu. Ihr Aufbau erinnert allgemein an neuronale Sprachmodelle (vgl. Almeida & Xexéo, 2023). Sie ermöglichen ein tieferes Verständnis von Sprache in Bezug auf Kontext und Satzbau, benötigen dafür jedoch mehr Rechenleistung und Daten (vgl. Hoque, 2024).

2.4 RAG-System (Mika Wettstaedt)

Retrieval-Augmented Generation (RAG) stellt ein hybrides Paradigma innerhalb der modernen Forschung zu Large Language Models dar. Grundlegend beschreibt RAG die Kombination von generativen Sprachmodellen mit externen Wissensquellen, um die Qualität und Faktentreue generierter Antworten zu verbessern. Während klassische LLMs ihr Wissen ausschließlich aus Trainingsdaten beziehen, erweitert RAG diesen Ansatz um die dynamische Komponente, Zugriff auf nicht-parametrisches Wissen zur Laufzeit zu haben. Dadurch wird eine klare Trennung zwischen Wissensspeicherung und Wissensverarbeitung ermöglicht, was insbesondere im Hinblick auf Aktualität und Skalierbarkeit von Bedeutung ist (vgl. Brown, et al., 2025 & Zhao, et al., 2026).

Das theoretische Grundprinzip von RAG basiert auf der Idee der Wissensaugmentation durch Retrieval. Formal lässt sich der Prozess als zweistufiges Modell beschreiben. Zunächst wird zu einer gegebenen Anfrage q eine Menge relevanter Dokumente D_k mittels einer Retrieval-Funktion bestimmt. Anschließend generiert ein Sprachmodell auf Basis der ursprünglichen Anfrage und der abgerufenen Dokumente eine Antwort y. Diese Entkopplung erlaubt es, das generative Modell flexibel mit unterschiedlichen Wissensbasen zu kombinieren, ohne dass ein erneutes Training erforderlich ist (vgl. Zhao, et al., 2026).

Im Zentrum von RAG-Systemen steht die Architektur, die typischerweise aus drei Hauptkomponenten besteht: dem Retriever, der Wissensbasis beziehungsweise dem Index sowie dem Generator. Diese Komponenten sind funktional voneinander getrennt, arbeiten jedoch eng zusammen, um eine kohärente und informationsbasierte Antwortgenerierung zu ermöglichen. Der Retriever übernimmt die Aufgabe, relevante Dokumente oder Textpassagen aus einer externen Wissensquelle zu identifizieren. Hierbei kommen unterschiedliche Retrieval-Ansätze zum Einsatz, die sich grundlegend in sparse und dense Methoden unterteilen lassen. Sparse Retrieval basiert auf klassischen informationsretrieval-basierten Verfahren wie BM25 und nutzt tokenbasierte Übereinstimmungen zwischen Anfrage und Dokumenten. Im Gegensatz dazu verwendet Dense Retrieval kontinuierliche Vektorrepräsentationen (Embeddings), um semantische Ähnlichkeiten im Vektorraum zu berechnen. Diese embedding-basierten Verfahren ermöglichen eine kontextsensitivere Suche und gelten als zentraler Bestandteil moderner RAG-Architekturen (vgl. Brown, et al., 2025).

Die Wissensbasis beziehungsweise der Index stellt die Datenquelle dar, auf die der Retriever zugreift. Diese kann sowohl strukturierte als auch unstrukturierte Daten enthalten, etwa Dokumentensammlungen, Webseiten oder domänenspezifische Textdatenbanken. Ein wesentlicher theoretischer Aspekt ist hierbei die Transformation der Daten in geeignete Repräsentationen, meist in Form von Vektorembeddings. Diese werden in spezialisierten Indexstrukturen

gespeichert, die eine effiziente Ähnlichkeitssuche ermöglichen. Die Qualität des Retrieval-Prozesses hängt dabei maßgeblich von der Repräsentation und Organisation dieser Wissensbasis ab (vgl. Neha, et al., 2025). Die dritte zentrale Komponente ist der Generator, in der Regel ein transformerbasiertes Sprachmodell. Dieses Modell verarbeitet sowohl die ursprüngliche Nutzeranfrage als auch die vom Retriever gelieferten Dokumente. Der zentrale Mechanismus besteht darin, die abgerufenen Informationen als Kontext in den Prompt des Modells zu integrieren. Dieser Prozess wird häufig als „context conditioning“ bezeichnet. Dadurch wird das Modell in die Lage versetzt, seine generierten Antworten auf konkrete externe Informationen zu stützen, anstatt ausschließlich auf internes, während des Trainings erlerntes Wissen zurückzugreifen (vgl. Klesel & Wittman, 2025).

Das Zusammenspiel dieser drei Komponenten kann entweder als Pipeline oder als stärker integriertes System realisiert werden. In der klassischen Pipeline-Architektur arbeiten Retriever und Generator sequenziell: Zunächst erfolgt der Abruf relevanter Dokumente, anschließend die Generierung der Antwort. Alternativ existieren Ansätze, die eine engere Kopplung der Komponenten anstreben, beispielsweise durch gemeinsames Training oder durch Feedbackmechanismen zwischen Retrieval und Generation. Diese unterschiedlichen Architekturen spiegeln verschiedene theoretische Ansätze wider, wie Wissen in generativen Systemen effizient integriert werden kann (vgl. Zhao, et al., 2026). Der Ablauf eines RAG-Systems folgt typischerweise einer klar strukturierten Sequenz von Schritten. Zunächst wird die Nutzeranfrage in eine geeignete Repräsentation überführt, häufig in Form eines Embeddings. Anschließend erfolgt der Retrieval-Schritt, bei dem die relevantesten Dokumente aus dem Index ausgewählt werden. Die Anzahl der berücksichtigten Dokumente (Top-k) stellt dabei einen wichtigen Parameter dar, der die Balance zwischen Informationsgehalt und Kontextlänge beeinflusst. Im nächsten Schritt werden die abgerufenen Dokumente zu einem Kontext zusammengefasst und gemeinsam mit der Anfrage an das generative Modell übergeben. Abschließend erzeugt das Modell eine Antwort, die idealerweise sowohl kohärent als auch faktisch korrekt ist (vgl. Li, et al., 2025).

Ein weiterer zentraler Aspekt betrifft die Wissensrepräsentation und die zugrunde liegenden Retrieval-Mechanismen. Moderne RAG-Systeme basieren in der Regel auf Vektorraummodellen, in denen sowohl Anfragen als auch Dokumente als hochdimensionale Vektoren dargestellt werden. Die Ähnlichkeit zwischen diesen Vektoren wird über Metriken wie Cosine Similarity oder das Skalarprodukt bestimmt. Diese mathematischen Grundlagen ermöglichen eine effiziente und semantisch fundierte Suche, sind jedoch auch mit Herausforderungen verbunden, etwa hinsichtlich der Skalierbarkeit oder der Sensitivität gegenüber feinen Bedeutungsunterschieden (vgl. Brown, et al., 2025).

Zusammenfassend lässt sich festhalten, dass die Architektur von RAG-Systemen eine modulare und theoretisch gut fundierte Grundlage für die Integration von externem Wissen in generative Modelle darstellt. Durch die klare Trennung von Retrieval und Generation sowie die Nutzung moderner Vektorrepräsentationen entsteht ein flexibles System, das sowohl anpassungsfähig als auch leistungsfähig ist. Gleichzeitig zeigt die Forschung, dass die Qualität der Gesamtausgabe maßgeblich vom Zusammenspiel der einzelnen Komponenten abhängt, insbesondere von der Leistungsfähigkeit des Retrievers und der Struktur der zugrunde liegenden Wissensbasis (vgl. Neha, et al., 2025 & vgl. Zhao, et al., 2026).

2.5 API-Schnittstelle (Mika Wettstaedt)

Die Nutzung von API-Schnittstellen stellt eine zentrale Grundlage für die in dieser Arbeit entwickelten KI-basierten Systeme dar. Eine API (Application Programming Interface) ermöglicht die standardisierte Kommunikation zwischen unterschiedlichen Softwarekomponenten. Sie definiert Regeln und Protokolle, über die Programme Anfragen senden und Antworten empfangen können, ohne die interne Funktionsweise des jeweils anderen Systems zu kennen. Im Kontext dieser Arbeit dient sie insbesondere dazu, externe Large Language Models in die eigenen Anwendungen zu integrieren, ohne diese lokal betreiben zu müssen (vgl. Park, et al., 2025). Im vorliegenden Projekt wurde die Chat-AI-API der Academic Cloud (SAIA – Scalable AI Accelerator) verwendet (vgl. GWDG, 2026). Diese API ist OpenAI-kompatibel aufgebaut und stellt somit eine standardisierte Schnittstelle zur Verfügung, über die verschiedene KI-Modelle angesprochen werden können. Der Zugriff erfolgt über einen individuellen API-Schlüssel sowie einen fest definierten Endpunkt. Für diese Arbeit wurden folgende Zugangsdaten bereitgestellt:

- API-Endpunkt: https://chat-ai.academiccloud.de/v1
- Authentifizierung: über einen persönlichen API-Key
- Zugriff auf verschiedene Modelle (z. B. Llama, Qwen, Mistral, DeepSeek)

Die API stellt mehrere zentrale Funktionalitäten bereit, die für die Umsetzung der beiden Ansätze relevant sind. Dazu gehören insbesondere v1/completions zur klassischen Textgenerierung, v1/chat/completions zur Verarbeitung dialogbasierter Eingaben und v1/models zur Abfrage verfügbarer Modelle. Für die in dieser Arbeit entwickelte Anwendung wurde primär die Chat-Completion-API verwendet, da sie eine strukturierte Kommunikation im Nachrichtenformat ermöglicht und somit besonders gut für komplexe Prompting-Strategien geeignet ist (vgl. GWDG, 2026).

Die technische Funktionsweise der API-Nutzung lässt sich vereinfacht in drei Schritte unterteilen: Zunächst wird eine Anfrage (Request) an den API-Endpunkt gesendet, die den gewünschten Modellnamen, die Eingabedaten (Prompts bzw. Nachrichten) sowie Steuerungsparameter wie Temperatur oder Token-Limit enthält. Anschließend verarbeitet das ausgewählte Modell die Anfrage auf einem externen Server. Abschließend wird die generierte Antwort (Response) im JSON-Format zurückgegeben und kann im eigenen System weiterverarbeitet werden. Die Integration der API in die eigene Anwendung erfolgte über das Python-Paket „openai“, wobei lediglich der API-Schlüssel und der Endpunkt angepasst werden mussten. Dadurch konnte die bestehende OpenAI-Struktur vollständig übernommen werden, was die Implementierung erheblich vereinfacht hat (vgl. Park, et al., 2025).

Ein wesentlicher Vorteil der API-basierten Architektur liegt in der Skalierbarkeit und Flexibilität. Neue Modelle können ohne Anpassung der Systemarchitektur genutzt werden, indem lediglich der Modellname ausgetauscht wird. Zudem entfällt der Bedarf an eigener Recheninfrastruktur, da die Verarbeitung vollständig serverseitig erfolgt. Gleichzeitig ergeben sich jedoch auch einige Herausforderungen. Dazu zählen insbesondere die Abhängigkeit von externen Diensten, mögliche Latenzen bei der Anfrageverarbeitung sowie sicherheitsrelevante Aspekte im Umgang mit API-Schlüsseln. Letztere müssen vertraulich behandelt und dürfen nicht öffentlich zugänglich gemacht werden. Aus diesem Grund wurde im Rahmen der Implementierung eine Auslagerung sensibler Daten in eine separate Umgebungsdatei (.env) vorgenommen (vgl. Chen, et al., 2015).

Zusammenfassend bildet die API-Schnittstelle die zentrale technische Grundlage für die Anbindung der verwendeten LLMs in beiden Ansätzen. Sie ermöglicht eine effiziente, standardisierte und skalierbare Nutzung moderner KI-Modelle und stellt damit eine entscheidende Voraussetzung für die automatisierte Anforderungsextraktion dar.

3 Zielsetzung (Pia Voigtländer)

Es gibt viele Möglichkeiten wie LLMs eingesetzt werden können, um bei der Generierung von Anforderungslisten zu unterstützten. In dieser Arbeit werden zwei mögliche Ansätze dafür verglichen. Bei dem ersten handelt es sich um eine Pipeline, welche für zwei Kernbestandteile jeweils ein LLM verwendet und deswegen doppelstufige LLM-Methode genannt wird. Bei dem zweiten Ansatz handelt es sich um das bereits in vielen Bereichen angewendete RAG-System. Beiden Ansätzen wird dieselbe Aufgabe gestellt. Sie sollen möglichst viele und nach INCOSE-Regeln korrekte Anforderungen erzeugen. Dafür bekommen beide Ansätze dasselbe Input-Dokument. Dabei handelt es sich um ein Kapitel aus einem Angebot (vgl. Anhang 1). Dieses Kapitel wurde explizit ausgewählt, da es neben Text sowohl Tabellen als auch Bilder, wie technische Zeichnungen enthält (vgl. Kapitel 4.1). Das Ergebnis dieser Aufgabe hängt bei jeglichem Ansatz von vielen Parametern ab. Darunter sind offensichtliche, wie die Menge und Qualität der ausgelesenen Informationen aus dem Input-Dokument, aber auch weniger offenkundige, wie das Chunking und die verwendeten Prompts. Um sicherzustellen das mögliche Unterschiede nicht auf diese Parameter zurückzuführen sind, werden diese bei den zu vergleichenden Ansätzen gleich gewählt (vgl. Kapitel 4.1). Somit sollten Unterschiede lediglich auf die verschiedenen Strukturen der Ansätze zurückzuführen sein.

Die Auswertung der generierten Anforderungslisten wird in zwei Teile aufgeschlüsselt. Zunächst wird eine generelle Bewertung durchgeführt. Die Kriterien dafür sind eine Auswahl aus den INCOSE-Regeln, welche für diesen spezifischen Anwendungsfall besonders interessant sind. Dazu gehört die Vollständigkeit, die Eindeutigkeit und die Atomarität. Zwei weitere Kriterien, welche betrachtet werden, ist die Redundanzfreiheit, um zu überprüfen, ob sich zwei Chunks widersprechen, und die inhaltliche Präzision zur Überprüfung des korrekten Auslesens. Der zweite Teil der Auswertung bezieht sich auf die lückenlose Rückverfolgbarkeit der Inhalte der erstellen Anforderungen auf eine Stelle im Input-Dokument. Dafür wird der Inhalt jeder Anforderung über den/die dazugehörigen Chunk/-s mit dem originalen Dokument abgeglichen. Der Grund für diese gesonderte Betrachtung ist, um zu ermitteln, ob die verwendeten LLMs in dem Prozess halluzinieren.

Schlussendlich wird analysiert welcher der beiden betrachteten Ansätze ein besseres Ergebnis erzeugt und ob es Unterschiede bei der lückenlosen Rückverfolgbarkeit der Inhalte gibt. Werden Unterschiede auftreten, so wird überprüft, was der Grund dafür sein könnte und mögliche Lösungsvorschläge gemacht.

4 Methodik

In dem Kapitel der Methodik werden zunächst die Voreinstellungen und Steuerungsparameter der beiden Programmieransätze vorgestellt. Die Voreinstellungen der Skripte sind für beide Ansätze gleich, um eine bessere Vergleichbarkeit im Hinblick auf die Leitfrage zu schaffen. Weiterführend werden jeweils die einzelnen Ansätze detailliert erläutert, um deren individuellen Funktionsweisen aufzuführen und diese später besser nachvollziehen zu können. Darauf wird sich primär bei dem ersten Ansatz auf das Python Skript *„OPT_V3_API_Chunking_Ansatz.py“* und beim zweiten Ansatz auf das Skript *„RAG-Ansatz.py“* bezogen (vgl. Anhang 5). Abschließend wird das Bewertungsschema präsentiert, welches zum anstehenden Vergleich und der Evaluierung der beiden Programmieransätze verwendet wird.

4.1 Voreinstellung der beiden Programmieransätze (Jan Ole Trosin)

Die erstellten Programmieransätze beruhen auf zwei unterschiedlichen Funktionsweisen zur detaillierten Textverarbeitung und Anforderungsextraktion. In dem ersten Ansatz wird eine doppelstufige LLM Methode implementiert, wodurch eine lineare Verarbeitung der Chunks hin zu ISO konformen Anforderungen gewährleistet werden kann. Beim zweiten Ansatz hingegen wird zur Verarbeitung der Chunks und der Anforderungsextraktion ein RAG System implementiert. Durch diese unterschiedlichen Herangehensweise ist es wichtig, vordefinierte Einstellungen und Funktionen im Skript zu verwenden, die im Anschluss eine effiziente Vergleichbarkeit schaffen. Aus diesem Grund wurden Funktionstypen, wie die Auslesung der Daten, das Chunking, die Auswahl der LLMs und weitere Funktionen identisch in beiden Skripten als Voreinstellungen implementiert, um den Vergleich nur auf die Datenverarbeitung durch das RAG System und der doppelstufigen LLM Methode beschränken zu können. Durch diese Vordefinierungen kann in der Bewertung und Diskussion eine fundierte Aussage über die Erfüllung der Systemanforderungen sowie über die Präzision beim Abgleich der Funktionsweisen, insbesondere im Hinblick auf die Textverarbeitung, getroffen werden.

Erste Voreinstellung: Multimodales PDF Parsing (Parallele Datenauslesung)

Die Datenauslesung des eingelesenen PDF Dokuments (vgl. Anhang 1) ist von hoher Bedeutung, denn nur durch eine detaillierte Erfassung aller relevanten Daten aus der Ursprungsquelle, kann eine effiziente Textverarbeitung in beiden Skripten erfolgen. Zu diesem Zweck wurde in beiden Skripten das multimodale PDF Parsing eingesetzt, bei dem in der Datenextraktionsphase sämtliche Datentypen, wie Fließtexte, Tabellen, technische Zeichnungen, Skizzen oder konzeptionelle Bilder, des eingefügten Pflichtenhefts mithilfe von zwei externen,

funktional unterschiedlichen Bibliotheken sowie durch den ergänzenden Einsatz eines Vision LLMs detailliert ausgelesen wurden (vgl. Abbildung 1).

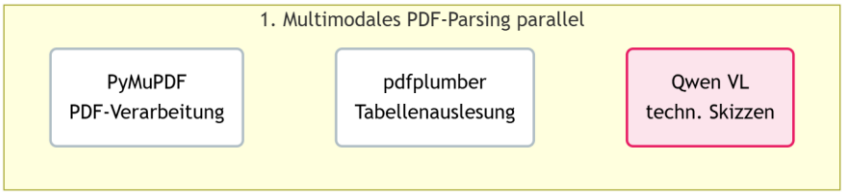


Abbildung 1: Parallele Bearbeitung aller Datentypen (Eigene Darstellung)

Die erste Bibliothek des multimodalem PDF Pasings nennt sich *PyMuPDF.* Der PDF Reader *PyMuPDF* gilt als eine sehr leistungsstarke Bibliothek, welche eine hohe Textverarbeitungsleistung und Rendering Qualität in der Datenauslesung verspricht. Die Bibliothek kümmert sich in den Skripten hauptsächlich um die Reinigung von eingelesenen Fließtexten aus dem PDF Dokument. Dabei werden alle falschen Zeilenumbrüche, mehrfache Leerzeichen und aussortierte Zahlen und Dokumentenhinweise, welche nicht für die Verarbeitung der aufgeführten Informationen relevant sind, herausgefiltert. Dies ermöglicht eine bessere Bereitstellung der einzeln, erfassten Textbausteine für das anschließende Chunking (vgl. Alkhamis, 2023, p. 28).

Als zweite Textbibliothek wurde der *pdfplumber* eingesetzt, wodurch ein zusätzlicher PDF Reader im Skript zur Verfügung gestellt wird, der insbesondere auf die Strukturierung und der numerische Extraktion von Daten spezialisiert ist. Die beiden verwendeten PDF Reader unterscheiden sich in ihren Leistungsprofilen. Wohingegen der *MyPuPDF* Reader primär für die Auslesung von Fließtexten optimiert ist, steht beim *pdfplumber* die Verarbeitung von numerischer Tabelleninhalte im Vordergrund. Bei dieser Auslesung zeigt der *MyPuPDF* jedoch noch Einschränkungen auf, wodurch der *pdfplumber* an dieser Stelle zusätzlich eingesetzt wird. Der *pdfplumber* ermöglicht die effizient und präzise Extraktion strukturierter Inhalte wie Tabellen, Koordinaten, Vektoren und Formeln (vgl. Sangeethasaravanan, 2025).

Durch die kombinierte Nutzung beider Bibliotheken erhöht sich die Genauigkeit der Datenauslesung, was vor allem bei der nachher anstehende ISO Anforderungsentwicklung von hoher Bedeutung ist, da diese nur auf den ausgelesen Informationen generiert werden dürfen. Ein weiterer Grund der Verwendung des *pdfplumbers* besteht insbesondere darin, dass die PDFs heutzutage aufgrund ihrer komplexen Formatstrukturen und ihrer primär visuellen Ausrichtung in der automatisierten Auslesung als herausfordernd gelten (vgl. Dong, et al., 2021). Dies wird vor allem bei gescannten, bildorientierten Dokumenten deutlich (vgl. Dong, et al., 2021). Im Kontext dazu, sind die Angebote häufig mit Systemmaßen und weiteren Kennwerten in tabellarischer oder visueller Form aufgeführt. Mithilfe des *pdfplumbers* können die Informationen

der einzelne Tabellenwerte gezielt in Sätze überführt werden, die anschließend als Chunks durch ein LLM als zusätzliche Information weiterverarbeitet werden.

Als letzte Instanz wird ein LLM eingesetzt, um technische Zeichnungen, Skizzen und Konzeptdarstellungen besser auslesen zu können. Hierbei wird das Vision LLM „qwen3-vl-30b-a3b-instruct“ verwendet, welches eine ähnlich Funktionsweise *pdfplumber* aufweist. „Qwen3-vl-30b-a3b-instruct“ kommuniziert über eine API Schnittstelle (vgl. Kapitel 2.5) mit einem externen Servern und arbeitete nach einer direkten Promptzuweisung. Im Skript wurde in beiden Fällen dafür der gleiche Code verwendet, um einen einheitliche Auslesung durch das LLM zu garantieren. Der Prompt lautet folgendermaßen:

> *„Du bist ein technischer Requirements Engineer, der das angehängte Bild einer PDF-Seite analysiert. Finde ALLE technischen Skizzen, Diagramme und Risszeichnungen. Extrahiere daraus ausschließlich die physikalischen Maße und technischen Werte des dargestellten Bauteils. WICHTIG: Beschreibe NIEMALS die Zeichnung selbst (vermeide Wörter wie 'Skizze', 'Zeile', 'Skala', 'Achse', 'Linie'). Formuliere nur die harten Fakten über das System (z.B. 'Der Abstand zwischen den Stützen beträgt 120 cm'). Ignoriere normalen Fließtext! Antworte nur, wenn Skizzen vorhanden sind, ansonsten antworte mit 'LEER'“* (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Mit diesem Prompt wird dem LLM die exakte Anweisung zur Auslesung der Daten gegeben. Der Prozess dahinter sieht folgendermaßen aus, dass nach der Verarbeitung des Prompts vom LLM, das LLM mehrere hochauflösende Bilder von den Skizzen, Zeichnungen und weiteren visuellen Datentypen erstellt. Diese Bilder werden dann in kleinere Segmente vom LLM zerlegt und Informationen daraus erzeugt. Danach formuliert das LLM die visuell erfassten Daten in Textform. Diese Textbausteine werden im Anschluss an das Skript zurückgegeben und für das anschließende Chunking bereitgestellt (vgl. Bai, et al., 2023, p. 3 ff.).

Durch die Kombination dieser drei Funktionstypen zur Datenextraktion können sämtliche relevanten Informationen aus dem bereitgestellten Angebot zuverlässig ausgelesen werden, wodurch eine hohe Präzision der Datenerfassung ermöglicht wird. Der identische Ablauf der Datenextraktion stellt dabei sicher, dass die Inhalte konsistent und vergleichbar aufbereitet werden. Auf dieser Grundlage können die extrahierten Daten einheitlich für das nachgelagerte, essenzielle Chunking vorbereitet und weiterverarbeitet werden.

Zweite Voreinstellung: Chunking

Auch für das anschließende Chunking wird in beiden Ansätzen dieselbe Methode angewendet. Mithilfe der externen Natural-Language-Processing (NLP) Bibliothek *spaCy* wird ein Chunking-Ansatz für die Textdaten angewendet, der vollständige, inhaltlich zusammenhängende Sätze

erkennt und den Text entsprechend segmentiert (vgl. Kshirsagar, 2024, p. 3 ff., & vgl. spaCy (a), kein Datum).

Die *spaCy* Funktionsbibliothek präsentiert eine Mischform den ursprünglichen Chunking Ansätzen, dem Recursive-Token-Chunker und des LLM-Semantic-Chunking Ansatzes. Wie in Kapitel 2.2 beschrieben, arbeitet der Recursive-Token-Chunker nach der Erkennung von Zeichen, welche einen Satz beenden (vgl. Amiri & Bocklitz, 2025). Nach diesen Kriterien teilt er die textuellen Daten ein und versucht damit die Chunks in einzelnen Sätze herunterzubrechen. Dies gestaltet sich jedoch gerade durch die Kommatrennung von Haupt- und Nebensätzen schwierig.

Zur Lösung derartiger Problematiken wird traditionell das LLM-Semantic-Chunking herangezogen, um Textdaten in sinnvolle Sätze zu unterteilen (vgl. Kapitel 2.2). Der Rückgriff auf ein vollständiges LLM erweist sich für diesen reinen Strukturierungsschritt in der Praxis jedoch als methodisch und technisch überdimensioniert. Das NLP-Chunking mittels spaCy fungiert hierbei als Bindeglied zwischen den genannten Strategien und stellt eine methodische Weiterentwicklung dar, die sich konzeptionell zwischen dem Recursive-Token- und dem LLM-Semantic-Chunking einordnen lässt. Dieser NLP-Ansatz basiert primär auf der statistischen Analyse linguistischer und syntaktischer Textmerkmale (vgl. spaCy (b), kein Datum). Dies ermöglicht eine präzise Identifikation von Satzstrukturen, sodass ausschließlich vollständige und inhaltlich kohärente Sätze als eigenständige Chunks extrahiert werden (vgl. spaCy (b), kein Datum).

Dies ist vor allem als Input für die Entwicklung von Systemanforderungen von hoher Bedeutung, da dadurch alle zusammenhängende Information, egal ob im Haupt- oder Nebensatz aufgeführt, erfasst und nicht abgetrennt werden. Das NLP-Chunking präsentiert damit eine intelligenten Chunking Ansatz, welcher gut aufbereitete Chunks zum Vorschein bringt, die durch weitere Informationsergänzungen, wie zum Beispiel einer Kontextualisierung, als Systemanforderung verarbeitet, werden können.

Dritte Voreinstellung: Verwendete LLMs

Auch die Auswahl der LLMs sind für beide Ansätze identisch. Es werden insgesamt drei unterschiedliche LLMs über die API Schnittstelle angesprochen, welche die weiterführenden Verarbeitung von bereitgestellten Informationen gewährleisten.

Das erste LLM ist das Vision Modell „qwen3-vl-30b-a3b-instruct“. Es wird zur Erfassung von visuellen Daten aus technischen Skizzen, Zeichnungen und Konzeptdarstellung im multimodalem Parsing verwendet. Qwen VL ist ein neuartiges Vision Modell, das speziell auf die Verarbeitung von graphischen Datentypen trainiert ist. Aufgrund seiner hohen Parameterverarbeitung von 30 Milliarden Parameter ist es ein sehr leistungsstarkes LLM. Der genaue Prozess

der Auslesung wurde schon im Vorfeld im Multimodalem PDF Parsing erläutert und kann dort nachgelesen werden (vgl. Bai, et al., 2023).

Für die Verarbeitung der Chunks (Generator Modell), hin zu ISO konformen Systemanforderungen, wird ein großes, textverarbeitungsbasiertes LLM verwendet. Das LLM nennt sich „llama-3.1-sauerkrautlm-70b-instruct“ und ist auf die Verarbeitung von deutschen Textstrukturen als Input spezialisiert. Aus diesem Grund ist zu beachten, sofern es zu einer Skalierbarkeit des Skriptes auf weitere Sprachen kommen sollte, dass ein anderweitiges Sprachprogramm auszuwählen ist, um die hohe Verarbeitungspräzision der jeweiligen Textchunks nicht zu beinträchtigen. Ein weiterer Vorteil bei der Anwendung des LLMs ist, sein großes Kontextfenster, welches die Verarbeitung umfangreicher Eingabedaten ermöglicht. In Kombination mit der hohen Modellkapazität von 70 Milliarden Parametern bringt dies eine intelligente Textlogik und ein tiefes Textverständnis mit, was für die korrekte Interpretation komplexer ISO-Richtlinien sehr wichtig ist (vgl. VAGO Solutions, 2024).

Das letzte verwendete LLM ist das „llama-3.3-70b-instruct“und dient als Überprüfungsmodell (Critic Modell) für die Evaluierung der INCOSE-Regeln. Während zuvor sprachliche Präzision gefragt war, benötigt diese Instanz reine logische Erfüllung der aufgeführten Prompt Anweisungen, um eine einzige Anforderung fehlerfrei gegen 42 INCOSE Regeln gegenzuprüfen. Durch die hohe Leistungsfähigkeit und das Textverständnis durch 70 Milliarden Parametern, präsentiert das LLM die nötige Kapazität, um große Regelwerke und weitere Informationen, welche für die Evaluierung und den Abgleich der INCOSE Regeln notwendig sind, detailliert verarbeiten zu können (vgl. meta-llama, 2024). Darüber hinaus erweist sich die hohe Zuverlässigkeit *(instruct)* als elementarer Vorteil, da das Modell garantiert, dass trotz der gewaltigen Prüflast die angegebene Formatstruktur für den abschließende Ausgabe sowie die Anweisungen für die Gegenprüfung strikt eingehalten werden.

Für alle verwendeten LLMs gelten in beiden Ansätzen die gleichen Einstellungen für die Temperatur und den Top P Wert. Die Temperatur eines LLM sagt aus, inwiefern kreative oder zufällige Antworten durch einen festgelegten Wahrscheinlichkeitswert für die nächste Antwortentwicklung berücksichtig wird (vgl. Verma, 2023). Der Top P Wert dagegen gibt die summierte Wahrscheinlichkeit des nächsten Worts für die Größe der Antwortentwicklung an (vgl. Verma, 2023). Hierbei wird festgelegt, wie groß das Vokabular in der kreativen und zufälligen Herleitung der Antwort sein soll (vgl. Verma, 2023). Dementsprechend beschäftigen sich beide Werte mit der kreativen Antwortgestaltung der Ausgaben.

Die Temperatureinstellungen aller LLMs liegt bei 0,0 und die Top P Werte bei 0,1. Dadurch sollen kreative Antworten hier vermieden und nur absolut wahrscheinliche Token in der Auswahl der Größe des Vokabulars verwendet werden, um möglichst präzise Antworten zu

erschaffen. Dies ist gerade im Requirements-Engineering sehr wichtig. Allerdings ist zu beachten, dass die gleiche Temperatureinstellung oder die Angabe von gleichen Top P Werten bei unterschiedlichen LLMs nicht zwangsläufig zu äquivalenten Ergebnissen führt und trotz konstanter Parameterwerte signifikante Unterschiede hinsichtlich der generierten Varianz und potenzieller Halluzinationen auftreten können (vgl. Li, et al., 2025 & vgl. Minh, et al., 2025). Aus diesem Grund wird im späteren Verlauf der Bewertung der Fokus auf die Rückverfolgbarkeit gesetzt, um das Verhalten von Halluzinationen nochmal gesondert zu betrachten.

Vierte Voreinstellung: INCOSE Regelwerk

Um nach der Chunkverarbeitung der jeweiligen LLMs einen ersten Messstandard zur Überprüfung der generierten Anforderungen in beiden Skripten zu implementieren, wurden die INCOSE-Regeln verwendet. Die Regeln werden als eigenes Skript in beiden Ansätzen vom Critic Modell ausgelesen. Nach der Auslesung durchläuft das Critic LLM mit der generierten Anforderung jede einzelne INCOSE-Regel und gleicht die Anforderungen mit dem Regelstandard ab. Dies dient der ersten Überprüfung der Konformität der Systemanforderungen.

INOCSE steht für International Council on System Engineering und repräsentiert ein internationales Forum für System Engineering (vgl. INCOSE Publications Office, 2023). Die INCOSE Regeln zeigen einen Leitfaden, nach dem ISO-konforme Anforderungen und Bedürfnisse in der Produktentwicklung detailliert und richtig formuliert werden können (vgl. INCOSE Publications Office, 2023). Insgesamt präsentiert der INOCSE Leitfaden 42 Regeln, die bei der Generierung von Anforderungen und Bedürfnissen berücksichtigt werden müssen. Dabei werden von inhaltlichen Notwendigkeiten bis hin zu formalen Formulierungsansätzen, alle Bedürfnisse und Anforderungen mit ihren jeweiligen Charakteristiken aufgezeigt und erläutert (vgl. INCOSE Publications Office, 2023).

Zur Veranschaulichung des INCOSE-Regelwerks zeigt das nachfolgende Beispiel eine Regel in ihrer codierten Form. Diese entstammt einem Python Skript, das vom Lehrstuhl Prof. A. Seibel für die Aufgabe „Pflichtenheftanalyse“ bereitgestellt wurde:

> *"R1": {"titel": "Strukturierte Anforderungen", "beschreibung": "Anforderungen sollen einer konsistenten Struktur folgen: [Bedingung] [Subjekt] [Modalverb] [Handlung] [Objekt] [Leistungsmerkmal].", "beispiel": "Wenn Benutzeranfragen verarbeitet werden, soll das Datenbanksystem Suchergebnisse innerhalb von 2,0 ± 0,5 Sekunden liefern."*}

(vgl. Anhang 5 *„INCOSE_REGELWERK.py“*)

Fünfte Voreinstellung: Nachweismethoden

Nach der Entwicklung und Prüfung der Anforderungen werden zudem durch das Critic LLM „llama-3.3-70b-instruct“ auch die Nachweismethoden für die jeweiligen Anforderungen

ermittelt. Hierbei wurden für beide Programmieransätze die gleichen Definitionen der Nachweismethoden Inspektion, Demonstration, Test und Analyse aus dem Vorlesungsskript verwendet (vgl. Anhang 5 *„Nachweismethoden_Definitionen_Analyse_Inspektion_Test_Demonstration_260208_210650.pdf“*). Diese wurden dem Critic Modell über einen auf die Ansätze angepassten Prompt mit weiteren Bewertungsbeispiel als Orientierung übergeben, um eine gleiche Bewertung der möglichen Nachweismethodik analysieren zu können und diese begründet, anhand der Anforderung darzulegen. Die Anweisungen im Prompt sind hierbei unterschiedlich und werden in den Kapiteln 4.2 und 4.3 näher erläutert, jedoch ist wichtig zu erwähnen das die Definitionen der Nachweismethoden in beiden Prompts gleich definiert sind.

Sechste Voreinstellung: Komprimierte PDF-Version

Um tatsächliche Ausgaben der beiden Ansätze miteinander vergleichen zu können, wird ein Pflichtenheft aus der Industrie als Eingabedokument verwendet. Grundlage der Datenextraktion bildet dabei eine gekürte Version des Pflichtenheftes, um den Rechenbedarf des ursprünglich 76 Seiten umfassenden Originaldokuments während der Aufbau- und Optimierungsphase der jeweiligen Skripte zu minimieren. Hierbei wurde bewusst darauf geachtet, dass der erstellte Auszug repräsentativ alle möglichen relevanten Informationsebenen (Fließtexte, Tabellen, Zeichnungen) beinhaltet, um eine ressourceneffiziente und gleichzeitig realitätsnahe Modellentwicklung zu garantieren. Da die implementierte Systemarchitektur dokumentenlängenunabhängig arbeitet, ist die volle Skalierbarkeit gegeben. Der Umfang zukünftiger Input-Dokumente schlägt sich lediglich in einer proportional erhöhten Rechenzeit und Systemauslastung nieder. (vgl. Anhang 1)

4.2 Erster Ansatz: Doppelstufige LLM-Methode (Jan Ole Trosin)

Die doppelstufige LLM-Methode umfasst fünf aufeinanderfolgende Verarbeitungsstufen. Das zugehörige Python Skript *„OPT_V3_API_Chunking_Ansatz.py“* sowie eine erläuternde Dokumentation *„README_DLLM.txt“* sind Teil der digitalen Abgabe, deren Ordnerstruktur in Anhang 5 abgebildet ist. Im Zentrum des ersten Ansatzes steht dabei die Transformation vorliegender Pflichtenheftdaten in ISO-konforme Anforderungen. Nachfolgend findet eine individuelle Evaluierung der erzeugten Anforderungen statt, um ihnen geeignete Nachweismethoden zuzuordnen. Die unterschiedlichen Verarbeitungsstufen sind in der Abbildung 2 aufgeführt und werden weiterführend vorgestellt.

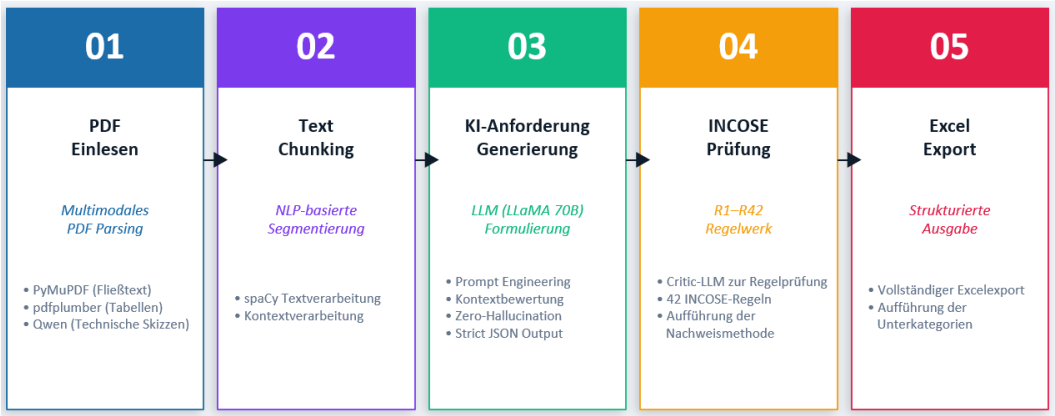


Abbildung 2: Unterteilte Verarbeitungsstufen der doppelstufigen LLM-Methode (Eigene Darstellung)

Die allgemeine Struktur des Programmiercodes orientiert sich an den Verarbeitungsstufen der doppelstufigen LLM-Methode aus der Abbildung 2. Bevor das Skript jedoch mit seiner eigentlichen Verarbeitung der Informationen aus dem Pflichtenheft startet, müssen die notwendigen Bibliotheken für die Verarbeitungsstufen importiert und die jeweiligen Konfigurationen der LLM Modell definiert werden. Aus diesem Grund startet das Python Skript mit dem Import der internen und externen Bibliotheken, um die Funktionsweisen der Verarbeitungsstufen mit den richtigen Importklassen sicherzustellen. Hierbei wird zwischen internen Standardbibliotheken der Pythonumgebung und externen, anwendungsbezogenen Bibliotheken unterschieden. (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Die Standardbibliotheken kümmern sich hauptsächlich um die Sicherstellung des technischen Fundaments des Ansatzes. Dabei wird die Dateninfrastruktur des Skripts aufgebaut und indirekt, die korrekte Handhabung der zu verarbeitenden Daten sichergestellt. Vor allem Themen wie die Textbereinigung und die Formatierungen der grundlegend, notwendige Python Zuweisungen werden hier betrachtet, um funktionstüchtige Interaktionen zwischen unterschiedlichen Verarbeitungsstufen und deren Funktionsweisen im Skript realisieren zu können. (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Zudem werden auch externe Bibliotheken zum Skript hinzugefügt, um die Intelligenz des Programms zu erweitern. Alle externen Bibliotheken sind in der Tabelle 1 aufgeführt.

Bibliothek	**Funktionsweise**
PyMuPDF (fitz)	Ermöglicht das detaillierte Auslesen von Fließtext & Bildern
Pandas	Datenstrukturierung für die Bereitstellung für den Excel Export
Tqdm	Visualisierung eines Fortschrittbalkens bei der AI Anwendung
Openai	Der OpenAI-API Client ermöglicht den Zugang zur externen Schnittstelle
spaCy	Semantische Trennung der Sätze auf Kontextbasis zur Unterstützung des Chunkings

pdfplumber	Auslesen von tabellenbezogenen Daten
openpyxl	Aufbereitung und Abspeicherung der Daten für die Pandas Bibliothek
dotenv	Verarbeitung von sensiblen Daten aus einer extern .env-Datei

Tabelle 1: Liste der extern importierten Bibliotheken für die dopplestufigen LLM Methode (Eigene Darstellung)

Die externen Bibliotheken ermöglichen die unterschiedlichen Funktionsweisen in den fünf Verarbeitungsstufen und Erweitern dabei die individuelle Handhabung von spezifischen Daten in komplexer Form. Hierbei sticht vor allem die NLP-Bibliothek *spaCy* heraus, welche zur detaillierten Textverarbeitung eingesetzt und vor allem im Funktionsabschnitt des Chunking verwendet wird. Weitere Bibliotheken und deren Funktionen werden weiterführend in der Erläuterung der jeweiligen Verarbeitungsstufen aufgeführt.

Nach dem Import der notwendigen Bibliotheken werden die generellen Konfigurationen im Skript definiert. Vor allem stechen dabei die einzelnen Konfigurationen für die verwendeten LLMs heraus. Die einzelnen LLMs werden über eine API-Schnittstelle, welche in Kapitel 2.5 näher erläutert wurde, angesprochen. Dieser Zugriff wird durch den Import der Bibliothek *openai* realisiert und erfolgreich implementiert, da die Bibliothek bei der Realisierung der Brücke zwischen dem internen Python Skript und des externen LLMs, angesprochen über den API Schlüssel, hilft (vgl. Thevapalan, 2025). Das bedeutet, dass die LLMs auf externen Servern laufen und nur über einen zur Verfügung gestellten Code des API Inhabers abgerufen werden können (vgl. Kapitel 2.5). Da es hier um vertrauliche Daten geht, wurde an dieser Stelle die Bibliothek *dotenv* eingesetzt. Diese Bibliothek ermöglicht die Erstellung eines zusätzlichen Datenblattes mit externen Umgebungsvariablen, die übergreifend aus dem Code angefragt und in das Python Skript übergeben werden können (vgl. Tuychlev, 2025). Dadurch ist der Code durch den Einsatz einer Variable verschleiert und für keinen anderen zugänglich (vgl. Tuychlev, 2025). Zudem werden für die Bedienung des Skripts noch weitere Parameter in der *.env* Datei definiert. Zum einen kann über die *.env* Datei die einzulesende Datei (Pflichtenheft) ausgewählt und die maximale Anzahl an Chunks definiert werden. Dies hilft, bei der Definition der zu verarbeitenden Dokumenten und der individuellen Einstellung der Rechenleistung, denn je mehr Chunks erzeugt werden, desto höher ist die Rechenleistung und die Dauer der Erzeugung des finalen Outputs.

Weiterführend, wie schon in Kapitel 4.1 erwähnt, werden für beide Ansätze die gleichen LLMs verwendet und die gleiche Einstellungsparameter gewählt. Dabei wir die Temperatur von 0,0 sowie der Top P Wert von 0,1 eingestellt. Diese Einstellungen bezwecken, dass nur deterministische und präzise Antworten generiert werden dürfen, die ohne jegliche kreativen Ergänzungen oder Halluzinationen ausgegeben werden sollen. Dabei ist jedoch zu beachten, dass

die Halluzination mit dieser Einstellung nicht abgeschaltet werden kann und damit auch nach der allgemeinen Bewertung, der Fokusbereich der Rückverfolgbarkeit (Traceability) beurteilt wird, um festzustellen, welche Ansätze eher anfällig für Halluzinationen sind.

Verarbeitungsstufe 1: PDF einlesen

Nach den Installationen der Bibliotheken und der Konfigurationen der LLMs wird mit der ersten Verarbeitungsstufe des Skripts gestartet, das Einlesen der PDF Datei. Hierzu findet ein Multimodales PDF Parsing statt. Dafür werden die Bibliotheken *PyMuPDF* und *pdfplumber* im Skript angesprochen, um die Bilder und Textbausteine aus dem einzulesendem PDF Dokument für das anschließende Chunking vorzubereiten. Wie schon in Kapitel 4.1 erwähnt, ist der Aufbau hier identisch mit dem des zweiten Ansatzes. Dies bedeutet, dass die gleichen Prozessschritte stattfinden, welche im Kapitel 4.1 für die erste Voreinstellung definiert wurden. Kurz gesagt, kümmert sich die *MyPuPDf* um die Reinigung des Fließtextes, der *pdfplumber* erfasst die numerischen und tabellarischen Werte und das Vision Modell „qwen3-vl-30b-a3b-instruct“ liest die visuellen Daten, durch eine direkte Prompt Anweisung (vgl. Kapitel 4.1) aus und gibt die Ergebnisse in Textform wieder zurück. Zusätzlich dazu, werden im Ansatz der doppelstufigen LLM-Methode die Kapitelüberschriften des analysierten Fließtextes als Banner für die weitere Textverarbeitung abgespeichert. Dies hilft dem LLM nachher, einen detaillierten Kontext für jeden Chunk pro Kapitel zu erstellen, die Herkunft der Informationen besser zuzuordnen und weitere indirekte Informationen aus kürzen Textabschnitten zu beziehen. Gerade hier ist das von enormer Bedeutung, da jeder einzelne Chunk analysiert wird und ohne weitere Informationen es zu einer Vielzahl von Ablehnung der daraus generierten Anforderungen kommen würde. Zudem wären dann die Chunks zu unpräzise und die Nachverfolgbarkeit der jeweiligen ausgelesen Daten wäre nicht gegeben. (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Verarbeitungsstufe 2: Text Chunking

In der zweiten Verarbeitungsstufe findet die Textzerlegung durch das Chunking statt. Dieser Schritt ist essenziell für die Input-Lieferungen der Anforderungsentwicklung und ist bei beiden Ansätzen erneut gleich aufgebaut (vgl. Kapitel 4.1).

Im Anschluss an das multimodale PDF-Parsing liegen die extrahierten Informationen als strukturierte Liste von Textbausteinen samt zugeordneter Kapitelüberschriften vor. Diese Datenstruktur wird zur weiteren linguistischen Verarbeitung an die NLP-Bibliothek *spaCy* übergeben. Die Funktion und Begründung der Auswahl dieser NLP-Bibliothek wurde zuvor in Kapitel 4.1 erklärt, da sie auf beide Anätze als zweite Voreinstellung definiert wurden, um eine gute Vergleichbarkeit zu schaffen. Jedoch kurz gesagt, trennt die *spaCy* Bibliothek die Texte in Sätze auf und verarbeitet sie damit zu einem Chunk. Die Bibliothek ist somit in der Lage zusammenhänge Sätze nicht nur an der Zeichensetzung, sondern auch an der inhaltlichen Konvergenz

zu erkennen und dementsprechend zu trennen. Zusätzlich dazu wurde zudem die Untergrenze von 20 Zeichen beim Chunking eingeführt, da davon ausgegangen wird, dass alle Textbausteine unter 20 Zeichen keinen aussagekräftigen Mehrwert liefern können und nur die Verarbeitung des Gesamtsystem verlangsamen. In dieses Muster fallen vor allem Seitenzahlen und Füllwörter, welche hierbei vernachlässigt werden.

Weiterführend werden die erzeugten Chunks für eine bessere Nachverfolgbarkeit mit weiteren Informationen, wie einer eigenen ID, dem Text an sich, dem davorstehenden Kontext, der Seitenzahl, dem Kapitelnamen und den weiteren Standortdetails in einer separaten Liste abgespeichert. Dies ermöglicht am Enden eine bessere Rückverfolgbarkeit der Chunks, um genau analysieren zu können, woher der Chunk stammt und ob das LLM wirklich nach der strikten Vorgabe, ohne Halluzinationen, gearbeitet hat.

Zusätzlich wird die Kontextbetrachtung für die jeweiligen Chunks definiert, um den vorherigen Input auch für inhaltsgeringere oder kürze Textchunks nutzen zu können. Gerade dadurch, dass bei diesem Ansatz immer nur ein Chunk verarbeitet wird, können durch die Bereitstellung des Kontextes mehr Informationen bereitgestellt werden, wodurch die Eindeutigkeit der Anforderungen gesteigert werden kann. Das LLM erhält damit weitere Details zur inhaltlichen Herkunft des Chunks für einen detaillierte Verarbeitung. Durch diese Modifikation entsteht ein gleitendes Fenster, das dem aktuell zu verarbeitenden Chunk die drei vorangegangenen Sätze als semantischen Kontext anfügt. Abschließend findet einen Duplikat Erkennung der jeweiligen erzeugten Chunks statt, um Mehraufwand für das LLM mit keiner aussagekräftigen Absicht zu vermeiden. (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Verarbeitungsstufe 3: KI-Anforderungen generieren

Der Fokus dieser Verarbeitungsstufe liegt auf der Transformation der extrahierten Chunks in ISO-konforme Anforderungen. Wie schon in Kapitel 4.1 beschreiben, wird hierfür das Generato Modell „llama-3.1-sauerkrautlm-70b-instruct“ verwendet, welches vor allem auf die Textverarbeitung deutscher Texte spezialisiert ist. Über die API-Schnittstelle wird der generierte Prompt sowie die zugehörigen Chunks zur Verarbeitung an das LLM übermittelt. Dabei erhält das Modell nicht nur den isolierten Textabschnitt, sondern auch dessen vorausgehenden Kontext. Um dabei eine effiziente Modellverarbeitung sicherzustellen, muss ein präziser und zugleich allgemeingültiger Prompt die genauen Rahmenbedingungen für das LLM definieren. Nur so führt die Auswertung der jeweiligen Chunks zu nachvollziehbaren und validierbaren Resultaten. Aus diesem Grund wurde der Prompt für das LLM strukturiert und instruktiv konzipiert, beginnend mit einer präzisen Definition der eigenen Rolle (gekürzte Version):

> *„Streng normativ arbeitender Requirements Engineer“*
> *„Absolute Bindung an das Regelwerk der ISO 29148:2018 und den INCOSE Guide“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“ & „*README_DLLM*“)

Weiterführend wird im Prompt das klare Ziel der erzeugten Anforderungen definiert, wonach das LLM arbeiten und handeln soll:

> *„Ableitung und Formulierung von präzisen, testbaren Systemanforderungen im strukturierten JSON-Format“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“ & „*README_DLLM*“)

Zudem werden die allgemeinen, strikten Regeln präsentiert, nachdem das LLM die ISO konformen Anforderungen für die jeweilig eingeführten Chunks entwickeln soll (gekürzte Version):

> *„Zero-Hallucination-Policy“*
> *„Vermeidung von Datenvermischung zwischen Kontext und eigentlichem Chunk“*
> *„9 verschärfte Kriterien als Leitplanken zur Anforderungsentwicklung“*
> *„Klare Strukturvorgaben der Anforderungen“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“ & „*README_DLLM*“)

Fortführend werden in dem Prompt zur Anforderungsentwicklung, die jeweiligen Ablehnungskriterien definiert. Dadurch ist das Modell in der Lage, nicht zielführende Eingaben zu identifizieren und zu markieren. Dies ist essenziell, da ein paar Chunks auch inklusive ihrer Kontextbetrachtung nicht genügend Informationen für die Generierung einer validen, ISO-konformen Anforderung aufweisen. Im Prompt werden dazu folgende Ablehnungskriterien definiert (gekürzte Version):

> *„Vage Formulierungen oder Begrifflichkeiten“*
> *„Flasche Zuordnungen der Anforderungen, sowie deren Datenbezüge“*
> *„Wenn die Eindeutigkeit der Anforderungen fehlt und Metadaten/Referenzen fehlen“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“ & „*README_DLLM*“)

Ergänzend zu den hier gekürzt, dargestellten Prompt Anweisungen werden dem LLM spezifische Beispiele übergeben. Durch die systematische Gegenüberstellung von Positiv- und Negativbeispielen erhält das LLM einen klaren Leitfaden für die Anforderungsextraktion. Dieser Ansatz befähigt das LLM, typische Fehler zu vermeiden, und richtet die Verarbeitung der Chunks optimal an den menschlichen Erwartungen aus. Nach erfolgreicher Verarbeitung werden die Chunks in einer separaten Liste gespeichert und mit den resultierenden Anforderungen verknüpft, um eine strukturierte Darstellung der Ergebnisse zu ermöglichen. Dieser Output im vorliegenden JSON-Format kann dann weiterverarbeitet werden.

Verarbeitungsstufe 4: INCOSE Prüfung

Nach der Entwicklung der ISO konformen Anforderungen aus den extrahierten Chunks folgt die kritische Beurteilung und Reflexion nach den vorgestellten INCOSE-Regeln. Wie in Kapitel 4.1 erläutert, stellen die INCOSE-Regeln einen Leitfaden zur fehlerfreien und präzisen Erstellung von ISO konformen Anforderungen auf. Die separat bereitgestellten INCOSE-Regeln, welche die Charakteristiken von insgesamt 42 Regeln umfassen, werden als codiertes Skript in die Python Umgebung eingelesen. Anschließend werden sie mithilfe einer spezifischen Prompt Anweisung an das Critic Modell „llama-3.3-70b-instruct“ zur individuellen Prüfung übergeben. Dieser Prozessschritt ist zudem für beide Ansätze gleich, um als Ergebnis ISO konforme Anforderungen zu gewährleisten. Dementsprechend sind die Einstellungen für das Critic LLM so umgesetzt, wie in Kapitel 4.1 im Vorfeld beschrieben.

Auch in diesem Verarbeitungsschritt wurde durch gezieltes Prompt Engineering sichergestellt, dass das LLM neben den Eingabedaten eine exakte Aufgabenstellung erhält. Der folgende, gekürzt dargestellte Prompt dient dem individuellen und systematischen Abgleich der generierten Anforderung mit jeder einzelnen INCOSE-Regel (gekürzte Version):

> *„Bewerte als INCOSE-Auditor die vorliegende Anforderung strikt nach den vorgegebenen Regeln (R1-R42) und ordne jeder Regel ausschließlich die Kategorien 'erfüllt', 'nicht erfüllt' oder 'nicht beurteilbar' zu. Verzichte dabei ausnahmslos auf jegliche Begründungen, Annahmen oder abweichende Formulierungen.“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“ & „*README_DLLM*“)

Nach der Überprüfung auf mögliche INCOSE Regelverletzungen werden die Ergebnisse der Anforderungen abschließend als „erfüllt“, „nicht erfüllt“ oder „nicht beurteilbar“ klassifiziert und in einer separaten JSON-Liste gespeichert. Eine Anforderung gilt hierbei als erfüllt, wenn es keine INCOSE-Regel gibt, die nicht erfüllt ist. Wird jedoch eine Regel nicht befolgt, durch eine individuelle Bewertung der Anforderung, dann wird sie als „nicht erfüllt“ markiert. Weiterführend kann es auch zu nicht beurteilbaren Anforderungen kommen. Diese bestehen meist aus unzureichenden Chunks, aus denen keine Anforderungen entwickelt werden konnten.

Zusätzlich dazu findet in der vierten Verarbeitungsklasse auch die Klassifizierung der jeweiligen Nachweismethoden statt. Hierzu wurden die Definitionen der jeweiligen Nachweismethoden Inspektion, Analyse, Demonstration und Test dem LLM für die Evaluierung und Zuordnung durch den Prompt an das Critic Modell übergeben (vgl. Anhang 5 *„Nachweismethoden_Definitionen_Analyse_Inspektion_Test_Demonstration_260208_210650.pdf“*). Die Definitionen sind wie in Kapitel 4.1 schon beschrieben, in beiden Ansätzen die gleiche, um eine gleiche Beurteilungsbasis bei der Klassifikation der Nachweismethoden pro Anforderungen zu

definieren. Der dazugehörige Prompt unterscheidet sich jedoch in den Ansätzen und ist auf den zu erwartenden Output angepasst. Die gekürzte Version der Prompt Anweisung lautet wie folgt:

> *„Du bist ein streng normativ arbeitender Requirements Engineer. Bestimme für die folgende Systemanforderung die am besten geeignete Nachweismethode.“*

(vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py“ & „README_DLLM*“)

Durch diese Prompt Anweisung gibt das Critic Model „llama-3.3-70b-instruct“ zu jeder Anforderung einen Nachweis aus und begründet die gewählte Methode kurz. Dies hilft im Umkehrschluss die Entscheidung des LLMs besser nachvollziehen zu können. Um die Entscheidungen und Begründung der gewählten Nachweismethode des LLMs für die eingefügten Anforderungen detailliert zu gestalten, wird erneut mit Beispielen gearbeitet. Sie dienen als Orientierung zur Beurteilung für das Critic Modell. Der erzeugte Output wird anschließend mit in der JSON-Liste der Ergebnisse abgespeichert, um diese JSON-Liste später in ein besseres Ausgabeformat zu konvertieren. (vgl. Anhang 5 „*OPT_V3_API_Chunking_Ansatz.py*“)

Verarbeitungsstufe 5: Excel Export

In der letzten Verarbeitungsstufe findet die Datenkonvertierung statt. Die Daten, die durch das Auslesen des PDF-Dokuments, dem aufbauenden Kontext, des Chunkings, der Anforderungsentwicklung, der INCOSE Bewertung, der Nachweismethodenbestimmung und der Nachweisbegründung erfasst wurden, sind in einer separaten JSON-Liste gespeichert. Abschließend wird die JSON-Datei mithilfe der Python Bibliotheken *pandas* und *openpyxl* in ein Excel-Format exportiert, um eine leserfreundliche Darstellung zu gewährleisten. Dabei transformiert *pandas* die JSON-Daten zunächst in eine tabellarische Struktur (CSV-Format) (vgl. McKinnney, 2010, S. 2) und nutzt anschließend *openpyxl* als Transformator, um den umgewandelten CSV-Datensatz im angefragten Excel-Format umzuwandeln und abzuspeichern (vgl. Gazoni & Clark, 2024). Dadurch erfolgt die Ausgabe des finalen Output an den Benutzer über das Terminal in Python. (vgl. Anhang 5 *„OPT_V3_API_Chunking_Ansatz.py“*)

Zusammenfassende Merkmale der doppelstufigen LLM-Methode

Im Allgemeinen betrachtet zeigt die doppelstufige LLM-Methode eine linearen Verarbeitungsansatz einzelner, erzeugter Chunks auf. Dementsprechend liegt der Fokus bei dem Ansatz auf der detaillierten Auslesung der Daten aus dem Pflichtenheft, woraus die einzelnen Anforderungen generiert werden können. Der im Anhang 2 separate Prozess Flow zeigt die geradlinige Verarbeitung sehr gut auf. Insgesamt zeigte sich jedoch bei der Optimierung, dass gerade durch den geradlinigen Verlauf, trotz des Einsatzes der API-Schnittstellen, ein lange Rechenzeit zum vollständigen Durchlauf des Skripts benötigt wird. Um diese Rechenzeit detailliert zu verfolgen wurde die Bibliothek *tqdm* verwendet, welche die Gesamtdauer immer mit einem

Fortschrittsbalken anzeigt und somit dem Benutzer ein direktes Feedback zur Ladedauer des Skripts über das Python Terminal übermittelt werden kann (vgl. Farnschläder, 2024). Dennoch entsteht bei der langen, schrittweisen Verarbeitung der Chunks in Kombination mit dem gleitenden Kontexterfassungsfensters ein hoher detaillierter Input. Dieser reichert die Datenbasis des LLMs mit zusätzlichem Kontext an, was sich positiv auf die Qualität der generierten Anforderungen auswirken könnte.

4.3 Zweiter Ansatz: Eigenes RAG-System (Irina Kireeva)

Im Kontrast zu der zuvor vorgestellten doppelstufigen LLM-Methode basiert der zweite Programmieransatz auf einem eigens entwickelten RAG-System (vgl. Kapitel 2.4). Dabei wird der Python Code als zusätzliche Dateien unter den Namen *„RAG-Ansatz.py“* mit einer weiteren *„README_RAG.txt“* Datei in Anhang 5 aufgeführt. Im Rahmen dieser Arbeit wird der RAG-Ansatz so definiert, dass die ausgelesenen Daten des Pflichtenhefts nicht linear und iterativ durch das Sprachmodell verarbeitet werden. Stattdessen werden sie zunächst in eine intelligente Wissensdatenbank in Form eines Vektor-Index überführt und dort dauerhaft gespeichert. Im anschließenden Prozess der Anforderungsgenerierung greift das System gezielt auf diese Datenbasis zu, indem es nach themenspezifischen Schlüsselwörtern sucht. Dabei werden ausschließlich die semantisch relevantesten Textbausteine als Kontext an das LLM übergeben. Gleichzeitig wird darauf abgezielt, redundante Anforderungen zu reduzieren und die Qualität der generierten Ergebnisse zu erhöhen.

Der Programmieransatz des RAG-Systems zeigt sechs unterschiedliche Verarbeitungsstufen auf, die als Verarbeitungsstufen durchlaufen werden müssen, um das komprimierte PDF-Dokument in validierte, ISO-konforme Anforderungen zu überführen (vgl. Abbildung 3). Da grundlegende Komponenten für die Vergleichbarkeit beider Ansätze identisch gehalten wurden, werden die jeweiligen Konfigurationen der LLM-Modelle sowie das strikte Regelwerk der INCOSE-Richtlinien als feste Module eingebunden (vgl. Kapitel 4.1).

Abbildung 3: Unterteilte Verarbeitungsstufen des RAG-Ansatzes (Eigene Darstellung)

Bevor die Hauptmerkmale des Ansatzes in Form von Verarbeitungsstufen umgesetzt werden können, müssen zunächst die erforderlichen Bibliotheken im Code definiert werden. Dabei wird im Programmiercode zwischen Standardbibliotheken der Python-Umgebung und zusätzlichen, anwendungsbezogenen Bibliotheken unterschieden. Zunächst werden grundlegende Standardbibliotheken wie *os, json, re* und *uuid* importiert, die das technische Fundament des Skripts bilden. Auffällig ist bei diesem Ansatz die ergänzende Nutzung der Bibliotheken *time* und *threading*. Diese sind erforderlich, um einen eigens entwickelten „Request Manager“ zu betreiben, der die asynchronen API-Aufrufe an die Sprachmodelle koordiniert und das System gezielt vor Überlastungen durch Rate Limits schützt.

Da das grundlegende Datenauslesen, das regelbasierte Chunking sowie der finale Excel-Export identisch zum ersten Ansatz umgesetzt sind, greifen beide Skripte auf eine gemeinsame Basis an Bibliotheken zurück (wie *PyMuPDF, spaCy, OpenAI, pdfplumber* und *pandas*). Diese wurden bereits in Kapitel 4.1 ausführlich erläutert und werden daher an dieser Stelle nicht erneut aufgeführt. Um die bestehende Architektur jedoch von einem reinen Textverarbeitungsskript in ein semantisches Suchsystem im Sinne eines RAG-Ansatzes zu überführen, wurden zusätzliche spezialisierte Bibliotheken integriert, die in Tabelle 2 dargestellt sind.

Bibliothek	Funktionsweise
sentence_transformers	Wandelt Text-Chunks in numerische Vektoren (Embeddings) um, damit das System den semantischen Kontext der Textbausteine mathematisch erfassen kann.
FAISS	Hochperformante Vektordatenbank (entwickelt von Meta). Speichert die generierten Embeddings und ermöglicht eine extrem schnelle semantische Ähnlichkeitssuche (Similarity Search) nach relevanten Textabschnitten.

numpy	Mathematische Basisbibliothek, die faiss und sentence_transformers bei der effizienten Verarbeitung und Speicherung der hochdimensionalen Vektormatrizen unterstützt.

Tabelle 2: Liste der RAG-spezifischen, extern importierten Bibliotheken (Eigene Darstellung)

Diese RAG-spezifischen Bibliotheken bilden das technische Rückgrat der zweiten und dritten Verarbeitungsstufe des Systems. Sie schaffen die notwendige Grundlage dafür, die erfassten Textbausteine effizient durchsuchbar zu machen und dem Sprachmodell kontextrelevantes Wissen gezielt und dynamisch zur Verfügung zu stellen.

Verarbeitungsstufe 1: Multimodale Datenerfassung & Vorverarbeitung

Nach der Initialisierung der Systemumgebung sowie der RAG-spezifischen Bibliotheken beginnt das Skript mit dem Einlesen der PDF-Datei. Zur Sicherstellung der methodischen Vergleichbarkeit mit dem doppelstufigen LLM-Ansatz wird dieselbe multimodale PDF-Parsing-Strategie angewendet (vgl. Kapitel 4.1). Der zentrale Unterschied liegt allerdings in der Zielsetzung der Datenverarbeitung. Während der doppelstufige LLM-Ansatz die Daten direkt für die lineare Weiterverarbeitung vorbereitet, dient die Datenerfassung im RAG-System ausschließlich der späteren Indizierung. Zur Sicherstellung der Rückverfolgbarkeit (Traceability) wird eine Funktion zur Seitenzahlextraktion (*extract_display_page_number*) implementiert, die Muster wie „XX/YYY“ erkennt und jedem extrahierten Element eine visuelle Seitenzahl als Metadatum zuordnet.(vgl. Anhang 5 *„RAG-Ansatz.py“*)

Verarbeitungsstufe 2: Segmentierung und Aufbau der Vektordatenbank

Nach der multimodalen Datenerfassung erfolgt in der zweiten Verarbeitungsstufe die Textzerlegung (Chunking). Die Umsetzung erfolgt analog zum doppelstufigen LLM-Ansatz, um die Vergleichbarkeit sicherzustellen (vgl. Kapitel 4.1). Im Unterschied zum linearen Ansatz werden die Chunks nicht direkt an das Sprachmodell übergeben, sondern zunächst in einer Vektordatenbank verarbeitet. Dieser Zwischenschritt umfasst die Umwandlung der Chunks in numerische Vektoren (Embeddings) mithilfe des *SentenceTransformer-Modells*, wodurch der semantische Kontext abgebildet wird. Danach werden die Embeddings in einem FAISS-Index (*IndexFlatL2*) gespeichert, um eine effiziente semantische Suche zu ermöglichen (vgl. Anhang 5).

Verarbeitungsstufe 3: Retrieval-Strategie und kontextuelle Anforderungsgenerierung

Die dritte Verarbeitungsstufe bildet die zentrale Logik des RAG-Systems: die gezielte Informationssuche und darauf aufbauende, kontextbasierte Anforderungsgenerierung. Im Gegensatz zum doppelstufigen LLM-Ansatz, der das Dokument linear verarbeitet, erfolgt hier eine themenbasierte Strukturierung. Grundlage ist ein Dictionary (TOPICS), das 14 Hauptmerkmale wie Geometrie, Kinematik oder Sicherheit jeweils mit spezifischen Schlagwörtern verknüpft

und damit die inhaltliche Navigation steuert (vgl. Anhang 4). Für jedes dieser Schlagwörter wird ein Retrieval-Prozess initiiert, der in drei klar getrennten Schritten abläuft: Zunächst werden die definierten Schlagwörter im Rahmen der Query-Formulierung als Suchanfragen verwendet und mit dem Embedding-Modell („all-MiniLM-L6-v2“) in Vektoren überführt. Dadurch befinden sich sowohl die Anfrage als auch die Datenbasis im selben semantischen Raum, was eine inhaltlich vergleichbare Repräsentation ermöglicht. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Anschließend erfolgt die Semantic Search, bei der der Query-Vektor mit den im FAISS-Index gespeicherten Vektoren verglichen wird. Die Ähnlichkeit wird dabei über die sogenannte L2-Distanz bestimmt, also die euklidische Distanz zwischen zwei Vektoren im mehrdimensionalen Raum. Konkret wird die Distanz berechnet, indem die Differenzen der einzelnen Vektorkomponenten quadriert, aufsummiert und anschließend die Wurzel gezogen wird. Je kleiner dieser Wert ist, desto ähnlicher sind sich die beiden Vektoren und damit auch die zugrunde liegenden Textinhalte. Auf diese Weise können semantisch relevante Textabschnitte identifiziert werden, ohne auf exakte Wortübereinstimmungen angewiesen zu sein.

Im dritten Schritt, dem Top-k Retrieval, werden die k relevantesten Chunks selektiert (hier k=15). Dieser Wert stellt einen pragmatischen Kompromiss dar, denn kleinere k-Werte führen zu einem sehr fokussierten, aber potenziell unvollständigen Kontext, wodurch relevante Informationen verloren gehen können. Größere k-Werte erhöhen hingegen die Kontextabdeckung, bergen jedoch das Risiko, irrelevante oder redundante Inhalte einzubeziehen und damit die Verarbeitung durch das Sprachmodell zu erschweren. Die Wahl von k=15 ermöglicht somit eine ausreichend inhaltliche Breite bei gleichzeitig kontrollierter Kontextgröße. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Fragenkatalog zur Reproduzierbarkeit der Ausgabe

Für die Erstellung des Lastenhefts wird der von Prof. Dr. Seibel und seinen Mitarbeitenden entwickelte und auf myStudy bereitgestellte Fragenkatalog in leicht angepasster Form genutzt (vgl. Anhang 5). Dabei werden Anforderungsmerkmale wie Geometrie, Sicherheit oder Kontrolle als Kategorie definiert und mithilfe von Beispielen aus dem vorliegenden Pflichtenheft eingegrenzt (vgl. Anhang 3). Durch eine Einbettung in den Code kann eine vergleichbare und reproduzierbare Ausgabe bei jedem Durchlauf gewährleistet werden.

Verarbeitungsstufe 4: Kontextuelle Anforderungsgenerierung (Augmented Generation)

In der vierten Verarbeitungsstufe erfolgt die eigentliche Anforderungsgenerierung und damit die Umsetzung des Retrieval-Augmented Generation Prinzips. Hier werden die zuvor selektierten Inhalte aus dem Retrieval mit der generativen Fähigkeit des Sprachmodells kombiniert. Wichtig ist dabei der sogenannte Context-Injection-Mechanismus. Die Top-15 Chunks aus der

vorherigen Verarbeitungsstufe werden zu einem konsolidierten, themenspezifischen Kontextblock zusammengeführt und direkt in den Prompt des Modells „llama-3.1-70B-Instruct“ eingebettet. Im Gegensatz zum linearen Ansatz erfolgt keine isolierte Verarbeitung einzelner Textbausteine, sondern eine ganzheitliche Bewertung des relevanten Wissensausschnitts. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Die Steuerung der Generierung erfolgt über einen strikt formulierten Prompt, der das Modell klar einschränkt. Wesentliche Vorgaben sind:

> *„Du bist Systems Engineer nach ISO 29148.“*
> *„NUTZE AUSSCHLIESSLICH die Informationen aus dem bereitgestellten Kontext. ERFINDE KEINE WERTE ODER ANFORDERUNGEN.“*
> *„Jede Anforderung MUSS zwingend mit 'Das System muss' beginnen.“*
> *„Wenn im Kontext KEINE konkrete Anforderung zu dem Thema [Keyword] steht, gib eine LEERE LISTE zurück: []“*

(vgl. Anhang 5 *„RAG-Ansatz.py“* & *„README_RAG.txt“*)

Durch diese Vorgaben wird das Modell gezwungen, seine Ausgabe direkt aus dem bereitgestellten Kontext abzuleiten. Fehlen passende Inhalte, erfolgt keine Generierung, wodurch Fehlinterpretationen vermieden werden. Gleichzeitig reduziert der aggregierte Kontext Redundanzen, da zusammenhängende Informationen gemeinsam bewertet werden. Die generierten Anforderungen werden abschließend strukturiert, im JSON-Format gespeichert und für die nachgelagerte Validierung bereitgestellt. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Verarbeitungsstufe 5: INCOSE-Regelprüfung und Self-Improvement-Loop

In der fünften Verarbeitungsstufe findet die kritische Beurteilung der zuvor generierten Anforderungen statt. Wie bereits im doppelstufigen Ansatz beschrieben, dient das strikte Regelwerk der ISO 29148 und des INCOSE-Guides hierbei als Bewertungsmaßstab (vgl. Kapitel 4.1). An dieser Stelle implementiert das RAG-System jedoch eine fundamentale methodische Erweiterung gegenüber der reinen linearen Verarbeitung: den Self-Improvement-Loop (*improve_requirement_loop).* Während der erste Programmieransatz nach der Regelprüfung lediglich eine statische Auswertung durchführt, nutzt das RAG-System die Ergebnisse der INCOSE-Prüfung aktiv, um Fehler eigenständig zu korrigieren. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Dieser iterative Verbesserungszyklus ist im RAG-System besonders relevant, da das LLM in Verarbeitungsstufe 4 auf einem zusammengefassten Kontext basiert und Anforderungen daher zwar inhaltlich korrekt, jedoch strukturell nicht ausreichend atomar formuliert sein können. Erkannte Regelverstöße werden extrahiert und gemeinsam mit der betroffenen Anforderung

erneut an das Sprachmodell übergeben. Dabei handelt es sich nicht um ein erneutes Retrieval der ursprünglichen Chunks, sondern um eine gezielte Nachbearbeitung der bereits generierten Anforderung. Das Sprachmodell greift in diesem Schritt also nicht erneut auf den vollständigen Kontext zu, sondern reformuliert die bestehende Anforderung unter expliziter Berücksichtigung der verletzten Regeln. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Der Verbesserungszyklus ist im Code auf maximal drei Iterationen begrenzt, um Endlosschleifen zu vermeiden und die Systemressourcen zu schonen. Wird die Anforderung innerhalb dieser Durchläufe erfolgreich korrigiert, wird sie als „perfekt formuliert“ markiert. Insgesamt führt dieser Ansatz zu einer signifikanten Qualitätssteigerung, da das System nicht nur Anforderungen generiert, sondern erkannte Schwächen eigenständig korrigiert. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Verarbeitungsstufe 6: Verifikationsplanung und Datenkonvertierung

Die sechste Verarbeitungsstufe ordnet die Verifizierungsmethoden zu und übernimmt den finalen Datenexport. Die Zuweisung der vier Methoden erfolgt analog zum doppelstufigen Ansatz (vgl. Kapitel 4.1). Im RAG-System erstellt das LLM zusätzlich auf Basis des bereitgestellten Validierungs-Kontexts einen schrittweisen Verifikationsplan (*„Step-by-Step“),* um die praktische Testbarkeit der Anforderung direkt sichtbar zu machen. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Zusammenfassende Merkmale des RAG-Ansatzes

Das RAG-System nutzt einen themenzentrierten, nicht-linearen Ansatz. Anstatt das Dokument chronologisch abzuarbeiten, extrahiert die Pipeline gezielt relevante Informationen für die Anforderungsgenerierung. Die Gesamtarchitektur der RAG-basierten Requirement Pipeline ist in Gänze dem Anhang zu entnehmen (vgl. Anhang 4). Wie im Flowchart ersichtlich, bündelt der RAG-Ansatz die zuvor erläuterten Verarbeitungsstufen in vier übergeordnete Prozessphasen. Besonders die Trennung von Off-line-Indizierung (Phase 1) und Online-Retrieval (Phase 2) sowie der zyklische Optimierungsprozess (Phase 3) unterscheiden sich methodisch deutlich vom linearen, doppelstufigen LLM-Ansatz. Daraus ergeben sich weiterführend zentrale Merkmale, die den RAG-Ansatz charakterisieren. Durch den Aufbau der Vektordatenbank in Phase 1 wird eine hohe Skalierbarkeit und Verarbeitungsgeschwindigkeit erreicht, da das LLM nicht jeden einzelnen Chunk sequenziell verarbeiten muss, sondern ausschließlich auf relevante Textgruppen im Rahmen des Top-k Retrievals zugreift, wodurch Laufzeit und API-Kosten insbesondere bei großen Dokumenten reduziert werden können. Gleichzeitig werden Redundanzen minimiert, da im Gegensatz zur linearen Verarbeitung einzelne Chunks nicht isoliert betrachtet werden, sondern durch die Context-Injection in Phase 2 eine ganzheitliche Bewertung

eines Themas erfolgt und dadurch überschneidungsfreie Anforderungen generiert werden. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

Darüber hinaus ermöglicht der Self-Improvement-Loop in Phase 3 eine autonome Qualitätssteigerung, indem erkannte Regelverstöße iterativ korrigiert werden und so die ISO-Konformität der Anforderungen verbessert wird. Gleichzeitig besteht eine starke Abhängigkeit von der Qualität des Retrieval-Prozesses, da die Genauigkeit der generierten Anforderungen direkt von der semantischen Suche abhängt. Werden keine passenden Chunks identifiziert, kann das LLM, bedingt durch die Zero-Hallucination-Policy, keine korrekte Anforderung ableiten, selbst wenn die Information im Dokument vorhanden ist. (vgl. Anhang 5 *„RAG-Ansatz.py“*)

4.4 Bewertungsschema (Raphael Wohlers)

Nachdem in den vorangegangenen Abschnitten die gemeinsame methodische Ausgangsbasis sowie die beiden Programmieransätze beschrieben wurden, folgt im nächsten Schritt deren Evaluierung. Ziel ist es, die von beiden Ansätzen generierten Anforderungen vor allem qualitativ zu vergleichen. Es soll also nicht allein erfasst werden, wie viele Anforderungen durch den jeweiligen Ansatz generiert wurden, sondern vielmehr, in welcher Qualität diese Anforderungen vorliegen. Hierfür wird im Folgenden ein Bewertungsschema eingeführt, das einen strukturierten Abgleich der KI-generierten Anforderungen mit einem menschlich erstellten Referenzkatalog ermöglicht. Im Mittelpunkt steht somit die Frage, inwieweit die erzeugten Anforderungen mit einer menschlich erstellten Referenz übereinstimmen und in welchem Maß sie zentrale Qualitätsmerkmale guter Anforderungen erfüllen.

Referenzbasis im Rahmen der Untersuchung

Als Referenzbasis für die Bewertung dient ein manuell erstellter Anforderungskatalog, der aus demselben Pflichtenheftabschnitt abgeleitet wird, welcher den beiden KI-Ansätzen zur Verfügung gestellt ist. Zu der Erstellung dieses Katalogs analysierten mehrere Bearbeiter den Dokumentenabschnitt unabhängig voneinander und formulierten die identifizierten Anforderungen in einer ISO/IEC-/IEEE 29148 orientierten Form (vgl. Anhang 5 *„Pflichtenheftanalyse_Anforderungen_Eigene_Bewertung.xlsx“*). Anschließend wurden die Einzelergebnisse verglichen und in einer gemeinsamen Masterversion zusammengeführt. Diese Endfassung fungiert im Rahmen der Untersuchung als Referenzbasis, also als menschlich erzeugte Vergleichsbasis für die Beurteilung der KI-generierten Anforderungen. Die Masterversion stellt dabei keine absolut objektive Wahrheit dar, sondern eine methodisch abgesicherte und intersubjektiv abgestimmte Referenz.

Bewertungsdurchführung

Die Bewertung der beiden Ansätze wird durch ein separates Bewertungsteam vorgenommen, das nicht identisch mit den Personen ist, die die jeweiligen Programmieransätze entwickeln.

Durch diese organisatorische Trennung sollte die Gefahr interessengeleiteter Selbstbewertung reduziert und die Nachvollziehbarkeit der Beurteilung erhöht werden. Grundlage der Bewertung ist die vorherig erwähnte und manuell erstellte Referenzversion. Dabei werden die KI Anforderungen darauf geprüft, ob sie sich einer Anforderung des Referenzkatalogs zuordnen lassen und in welchem Maß sie die, folgend noch ausgearbeiteten, Bewertungskriterien erfüllen.

Bewertungskriterien

Zur Bewertung der KI-generierten Anforderungen wurden sieben Qualitätskriterien herangezogen. Die Auswahl orientiert sich an den etablierten Merkmalen guter Anforderungen in der Requirements Engineering Literatur (vgl. Zowghi & Gervasi, 2002) sowie an der Norm ISO/IEC-/IEEE 29148 (vgl. International Organization for Standardization, 2018). Folgende Kriterien wurden diesbezüglich herauskristallisiert:

Kriterium	Erklärung
Rückverfolgbarkeit	Die Anforderung bezieht sich eindeutig auf eine konkrete Aussage oder Textstelle des Pflichtenheftes und lässt sich einwandfrei auf diese zurückführen.
Vollständigkeit	Die Anforderung enthält alle für ihre Aussage notwendigen Bestandteile und es fehlen somit keine wesentlichen Informationen aus dem Ursprungsinhalt.
Eindeutigkeit	Die Anforderung ist interpretationsarm formuliert.
Redundanzfreiheit	Die Anforderung ist nicht doppelt vorhanden oder ist nicht überlappend mit einer anderen Anforderung
Atomarität	Die Anforderung beinhaltet einen klar Abgrenzbaren Sachverhalt bzw. eine klar abgrenzbare Forderung.
Inhaltliche Präzision	Die Anforderung gibt den Inhalt des Pflichtenheftes semantisch korrekt wieder ohne Informationen zu verfälschen oder unbegründet zu erweitern.
Validierungsmethode	Der Anforderung wird eine passende Validierungsmethode zugeordnet.

Tabelle 3: Allgemeine Bewertungskriterien für die ISO konforme Anforderungen

Begründung der Methodenauswahl

Die Wahl des Vorgehens erfolgte, da ein rein quantitativer Vergleich der generierten Anforderungen nicht ausreicht, um die Leistungsfähigkeit der beiden Ansätze angemessen zu beurteilen. Für die Bewertung KI-generierter Anforderungen ist nicht nur relevant, ob die Anforderungen grundsätzlich erkannt werden, sondern auch, ob diese inhaltlich korrekt, eindeutig, vollständig, nicht redundant und prinzipiell prüfbar formuliert sind. Der Einsatz eines menschlich erstellten Referenzkatalogs ermöglicht hierfür einen strukturierten Soll-Ist-Abgleich zwischen den KI-generierten Anforderungen und einer konsolidierten, manuell abgeleiteten Vergleichsbasis. Das gewählte Vorgehen ist daher geeignet um Unterschiede zwischen den KI-Ansätzen sowie Unterschiede zur manuell bearbeiteten Fassung aufzuzeigen.

Auf den Einsatz eines Reliabilitätsmaßes wie *Krippendorffs Alpha* wurde in dieser Arbeit bewusst verzichtet. Ein solches Maß dient primär der statistischen Erfassung der Übereinstimmung mehrerer Bewertender bei der Zuordnung identischer Untersuchungseinheiten zu klar definierten Kategorien (vgl. Krippendorff, 2011). Im vorliegenden Fall stand jedoch nicht die Messung dieser Form der Reliabilität im Vordergrund, sondern der qualitative Vergleich zweier KI-Ansätze. Im folgenden Kapitel wird das herausgearbeitete Bewertungsschema auf die Ergebnisse beider Ansätze angewendet, um deren Stärken und Schwächen zu analysieren.

5 Ergebnisse

Nachfolgend sollen die Outputs beider Lösungsansätze anhand der zuvor vorgestellten Kriterien bewertet werden. Die einzelnen Aspekte der Bewertung müssen außerdem bezüglich ihrer tatsächlichen Aussagekraft über die Eignung der Ansätze eingeordnet werden. Hierbei wird insbesondere bei festzustellenden Schwächen zwischen zwei Arten unterschieden, nämlich zwischen Schwächen und Unzugänglichkeiten, die auf die Umsetzung des Ansatzes zurückzuführen sind und jener, die auf die Struktur des übergeordneten Ansatzes zurückzuführen sind. Dadurch wird gewährleistet, dass nicht die durch die Verfasser dieser Arbeit realisierte Umsetzung der Ansätze, sondern die Ansätze selbst verglichen werden. Diese Einordnung der Bewertung findet im nachfolgenden Kapitel 6 statt.

Da die Bewertung aus in Kapitel 4.4 beschriebenen Gründen qualitativ, statt quantitativ, durchgeführt wird, werden zur Erläuterung verschiedener Erkenntnisse beispielhafte Funde aus den Ergebnisdateien *„FINAL_ISO_29148_Requirements_Ergebnis_Doppelstufiges_LLM.xlsx“* für den ersten Ansatz und *„ISO_29148_Requirements_Ergebnis_RAG.xlsx“* für den zweiten Ansatz aufgeführt (vgl. Anhang 5). Eine detailliertere Analyse ist jeweils den beiden Ergebnisdateien im Arbeitsblatt „Bewertung“ zu entnehmen.

5.1 Bewertung der Ergebnisse – Erster Ansatz (Luca Russek)

Als erstes wird der Output der doppelstufigen LLM-Methode bewertet, welcher im Anhang 5 in der Ordnerstruktur im Ordern des ersten Ansatzes unter dem Dateinamen *„FINAL_ISO_29148_Requirements_Ergebnis_Doppelstufiges_LLM.xlsx“* zu finden ist. Einleitend lässt sich festhalten, dass durch die Anwendung der bereits validierten INCOSE-Regeln erwartungsgemäß, grundsätzlich ISO-konforme Anforderungen generiert werden.

Schwächen zeigen sich jedoch vereinzelt in der Atomarität der formulierten Anforderungen. Beispielhaft hierfür sind die aus Seite 2 des Input-Dokuments generierten Anforderungen bezüglich der Compartment-Ebenen. Für jede Ebene wird eine Anforderung formuliert, die sowohl die lichte Höhe als auch die Höhe der Oberkante beinhaltet. Für eine saubere Atomarität müssten diese beiden Maßangaben jedoch in zwei separate Anforderungen aufgeteilt werden. Entsprechende Schwächen bezüglich Atomarität treten immer dann auf, wenn ein einzelner Chunk mehr als eine Anforderung enthält.

Weiterhin zeigt sich beim Abgleich mit der manuell erstellten Anforderungsliste, dass der Output nicht vollständig ist und an mehreren Stellen keine Eindeutigkeit aufweist. Während aus Text-Chunks nahezu alle Anforderungen, die auch durch manuelle Auswertung identifiziert

wurden, in vergleichbarer Eindeutigkeit extrahiert werden, treten Schwächen insbesondere bei Abbildungen und Tabellen auf.

Beispielhaft hierfür sind die aus Seite 2 des Input-Dokuments abgeleiteten Anforderungen. Tabellen werden zwar in Text-Chunks überführt, verlieren dabei jedoch offenbar Teile ihres Kontextes, insbesondere Überschriften. Dadurch sind die für Kapitel 4.1.1.2 des Pflichtenheftes generierten Anforderungen nicht eindeutig genug, da die klare Zuordnung der Werte zum Kontext (z. B. Compartment 1) fehlt. Infolge des fehlenden Kontextes wird zudem derselbe Inhalt für Compartment 2 und 3 nicht berücksichtigt und fehlt somit vollständig im Output. Dasselbe tritt auch bei den für Kapitel 4.1.5 des Input-Dokuments ausgegebenen Anforderungen auf. Auch hier geht der Kontext der Abbildungen verloren, wodurch die Anforderungen weder vollständig noch eindeutig formuliert sind.

Die Bedeutung der Kontextberücksichtigung zeigt sich auch bei den aus Kapitel 4.1.6 und 4.1.7 des Pflichtenheftes generierten Anforderungen. Hier wird im Output nicht erkannt, dass es sich um Anforderungen an den Auftragsteller und nicht an die umsetzende Partei handelt. Der entsprechende Kontext wird nicht berücksichtigt, wodurch Anforderungen formuliert werden, die für das Lagerlogistikunternehmen nicht relevant sind.

Bezüglich der Redundanzfreiheit lässt sich eine weitere Schwäche bei der Verarbeitung von Tabellen beobachten. Für Kapitel 4.1.1.4 des Pflichtenhefts wurden mehrere Anforderungen mehrfach formuliert. Grund hierfür ist, dass die Tabelle, deren Chunks als Grundlage für diese formulierten Anforderungen genutzt wurden, hauptsächlich aus Text besteht. Dies führt dazu, dass der Inhalt einmal als Text betrachtet wird und trotzdem auch als Tabelle, die wiederum in Textchunks umgewandelt wird. Somit sind die Chunks doppelt vorhanden und entsprechend doppeln sich für dieses Kapitel auch die Anforderungen.

Positiv hervorzuheben ist hingegen die Zuordnung geeigneter Validierungsmethoden. Unabhängig davon, ob Anforderungen korrekt und vollständig extrahiert wurden, wird ihnen nahezu durchgehend eine passende Validierungsmethode zugeordnet, die weitgehend mit der manuell erstellten Anforderungsliste übereinstimmt. Auch die inhaltliche Korrektheit zwischen Chunk-Inhalt und dem Inhalt der tatsächlichen Anforderung ist umfassend gegeben. Hier treten keine Halluzinationen auf.

Ein weiterer positiver Aspekt liegt in der zuvor bereits als nicht einwandfrei beschriebenen Vollständigkeit. Durch den doppelstufigen LLM Ansatz wird jeder Chunk des Input-Dokuments tatsächlich berücksichtigt, wodurch zumindest das Potenzial für eine ganzheitlich vollständige Anforderungsliste besteht, sofern der Kontext aus Tabellen und Abbildungen optimaler mit einbezogen werden kann. Inwieweit die Mängel bezüglich der Vollständigkeit auf Limitationen

des Ansatzes selbst, oder nur dessen Umsetzung zurückzuführen sind, wird wie zuvor erwähnt im folgenden Kapitel betrachtet.

Abschließend wird die Rückverfolgbarkeit betrachtet. Dabei wird sowohl die Rückverfolgbarkeit von der formulierten Anforderung zum Original-Chunk als auch vom Original-Chunk zum Ausgangsdokument bewertet. Für jede Anforderung werden der zugrunde liegende Chunk sowie die drei jeweils vorhergehenden Chunks angegeben. Diese Zuordnung ist umfassend akkurat, sodass sich die formulierten Anforderungen konsistent auf die Inhalte der entsprechenden Chunks zurückführen lassen. Auch die Lokalisierung des Chunks im Originaldokument ist akkurat. Hierfür wird sowohl die Seitenzahl als auch die Kapitelnummer angegeben. Verbesserungswürdig ist hier lediglich die Handhabung von Tabellen und Abbildungen. Tabellen werden im Ergebnisdokument nummeriert, erlauben jedoch keinen Rückschluss darauf, welche Tabelle aus dem Originaldokument referenziert wird, sofern auf einer Seite in einem Kapitel mehrere Tabellen vorhanden sind. Ähnlich verhält es sich mit Textchunks, die aus Abbildungen erstellt wurden. Hier fehlt in der Ergebnisdatei zusätzlich eine Nummerierung.

5.2 Bewertung der Ergebnisse – Zweiter Ansatz (Lucas Russek)

In diesem Kapitel werden die Ergebnisse des RAG-basierten Ansatzes bewertet und stüzen sich auf die Ergebnisdatei *„ISO_29148_Requirements_Ergebnis_RAG.xlsx“* in Anhang 5. Ähnlich, wie beim ersten Ansatz lassen sich auch für den zweiten Ansatz Mängel bezüglich der Vollständigkeit und Eindeutigkeit feststellen. Diese sind hier noch stärker vertreten als noch beim ersten Ansatz. Während beim ersten Ansatz Teile von Unterkapiteln nicht berücksichtigt werden, lässt der zweite Ansatz ganze Kapitel aus. Die meisten Anforderungen entstammen den Seiten 7-10 des Inputdokuments (vgl. Anhang 1). Von den anderen Seiten werden die in der manuell erstellten Anforderungsliste als relevant erkannte Anforderungen kaum, beziehungsweise gar nicht, extrahiert. Des Weiteren fällt auf, dass nicht jeder Chunk des Input-Dokuments tatsächlich zur Bildung einer Anforderung in Betracht gezogen wird.

Auch die Eindeutigkeit ist nicht einwandfrei gegeben. Jede Anforderung wird mit „Das System...“ eingeleitet, hierdurch sind Anforderungen nicht genauen Bereichen und Teilsystemen zuzuordnen. Im Ergebnis werden somit zum Teil identisch formulierte Anforderungen mit verschiedenen Werten formuliert, da der Kontext der Fachlasten auf Seite 7-10 des Input-Dokuments nicht berücksichtigt wird. Es gibt somit beispielsweise mehrere „Höhenangaben“ für „das System“, während eine Einordnung, um welchen Regalfachtyp bei bestimmter Last und Lastverteilung auf eine bestimmte Anzahl an Stellplätze es sich handelt, ausbleibt.

Analog zum ersten Ansatz wird auch hier der Kontext der letzten beiden Seiten nicht erkannt und Anforderungen formuliert, die keine Anforderungen an den Logistikunternehmer

darstellen. Was sich jedoch nicht mit dem ersten Ansatz deckt, ist, dass im Ergebnis vom zweiten Ansatz eine Halluzination festgestellt werden kann. Hier wird die inhaltliche Korrektheit vom Übertrag des Chunk-Inhalts in eine Anforderung verletzt. Konkret tritt die Halluzination bei der Anforderung in Reihe 46 der Ergebnisdatei auf. Die hier angegebenen 95% entstammen keinem der zugrundeliegenden Chunks und sind nach weiteren Nachprüfungen auch nicht an anderer Stelle weder dem Input-Dokument noch der manuell erstellten Anforderungsliste zu entnehmen.

Die Redundanzfreiheit scheint erst einmal auch quantitativ bewertbar zu sein, hier muss jedoch berücksichtigt werden, dass doppelt formulierte Anforderungen potenziell nicht zweimal dieselbe Anforderung beschreiben, sondern verschiedene Anforderungen mit in Teilen gleichen Werten, bei denen die oben beschriebenen Mängel eigentlich in der Eindeutigkeit liegen, da diese z.B. nicht verschiedenen Fächertypen zugeordnet sind. Im Ergebnis liegen fünf Anforderungen vor, die entweder identisch oder im Inhalt nicht voneinander unterscheidbar sind. Beispielhaft hierfür sind die Anforderungen aus Reihe 10 und Reihe 16 im Ergebnisdokument.

Wie beim ersten Ansatz, ist auch für den RAG-basierten Ansatz die Zuordnung einer Validierungsmethode als gelungen zu bewerten. Unabhängig davon, ob Anforderungen korrekt und vollständig extrahiert wurden, wird ihnen nahezu durchgehend eine passende Validierungsmethode zugeordnet, die weitgehend mit der manuell erstellten Anforderungsliste übereinstimmt.

Zuletzt wird die Rückverfolgbarkeit betrachtet. Den verwendeten Chunks ist analog zum ersten Ansatz eine Seitenzahl zugewiesen, diese Zuweisung ist vollständig korrekt. Die Zuweisung des Kapitels fehlt hingegen bei zweitem Ansatz.

Die Rückverfolgbarkeit von der generierten Anforderung zum basierenden Chunk ist jedoch nur eingeschränkt möglich. Es werden für jede erzeugte Anforderung 15 Chunks angegeben, jedoch ist nicht eindeutig zu identifizieren, welcher Chunk bzw. welche Chunks den maßgeblichen Inhalt für die Anforderung liefern und welche Chunks potenziell „nur“ zum Auffüllen der 15 Chunks hinzugefügt werden, ohne den Inhalt der Anforderung selbst zu stützen.

6 Diskussion & Einordnung (Luca Russek)

Die in Kapitel 5 durchgeführte Bewertung der Ergebnisse der verschiedenen Ansätze soll als Grundlage für einen direkten Vergleich des doppelstufigem LLM-Ansatzes und RAG-basiertem Ansatz dienen. Damit im Zuge dieses Vergleiches tatsächlich die Ansätze selbst und nicht ihre Umsetzung durch die Verfasser dieser Arbeit verglichen werden, müssen die Ergebnisse der Bewertung zuvor eingeordnet werden. Es gilt zu beachten, dass folgende Einordnungen keinen absoluten Wahrheitsanspruch erheben. Es handelt sich um Einschätzungen und Lösungsideen der Verfasser dieser Arbeit. Um diese zu jedoch zu validieren, bedarf es im Anschluss an diese Arbeit weiterer Forschungsaufwände. Zu Beginn werden die festgestellten Schwächen von ersten Ansatz betrachtet.

Die Limitation bezüglich der Atomarität tritt explizit auf, wenn in einem Chunk mehrere Anforderungen beinhaltet sind. Es liegt nahe, dass dies auf eine suboptimale Umsetzung zurückzuführen ist, die durch Integration einer expliziten Logik in den Code behoben werden kann. Mögliche Ansätze können hierfür ein Loop, der bei verletzter Atomarität Anforderungen separiert, oder einfaches optimieren des Codes, sodass direkt mehrere Anforderungen aus einem Chunk generiert werden können, sein.

Die nicht einwandfreie Vollständigkeit und Eindeutigkeit kann möglicherweise auf den Kontextverlust bei Tabellen und Abbildungen zurückgeführt werden. Auch dies deutet auf eine Schwäche in der Umsetzung hin. Konkret werden Chunks mit weniger als 20 Zeichen nicht betrachtet. Chunks dieser Größe repräsentieren häufig Überschriften und Unterüberschriften wie „Compartment 1“, genau der der Kontext, der in Kapitel 5.1 als fehlend herausgestellt wurde. Da der Ansatz nicht aus jedem Chunk zwingend eine Anforderung generiert liegt zur Optimierung nahe, auch Chunks mit einzubeziehen, die weniger als 20 Zeichen beinhalten. Auch erscheint es hilfreich, den durch die Überschrift einer Tabelle hinzugefügten Kontext in jeden Textchunk, der aus der entsprechenden Tabelle erzeugt wurde, mit aufzunehmen.

Die aus Kapitel 4.1.6 und 4.1.7 des Pflichtenheftes fälschlicherweise extrahierten Anforderungen deuten darauf hin, dass der Lösungsansatz auch hier nicht den Kontext erkennt, in dem der Inhalt steht. Hier muss nachfolgend geprüft werden, inwieweit eine Optimierung in der Umsetzung des Ansatzes ermöglicht, genug Kontext durch zum Beispiel einen Loop, der Anforderungen auf ihre Relevanz für Auftragnehmer oder Auftraggeber prüft, einzubringen. Speziell bei komplexeren Angeboten, bei denen dieser Kontext unter Umständen gar nicht im betrachteten Chunk selbst auftaucht, kann der betrachtete Ansatz selbst potenziell eine strukturelle Limitation aufweisen. Wie zuvor beschrieben bedarf dies weiterer Nachforschung.

Das Auftauchen doppelter Anforderungen hingegen erscheint klar auf die Umsetzung zurückzuführbar zu sein. Hier kann ein simpler Loop integriert werden, der jede generierte Anforderung mit den vorherig generierten Anforderungen auf Redundanzen abprüft.

Abschließend können auch die Limitationen der Rückverfolgbarkeit potenziell durch Verbesserung der Umsetzung behoben werden. Ein geeigneter Lösungsansatz ist, die Tabellen mit Titeln zu versehen, anstatt diese zu nummerieren. Dies ist durch kleine Anpassungen im Code leicht zu realisieren.

Die meisten Limitationen des ersten Ansatzes scheinen auf die Umsetzung zurückführbar und somit potenziell behebbar zu sein. Ebendiese Einordnung wird nachfolgend für den zweiten Ansatz durchgeführt.

Die einwandfreiere Atomarität lässt sich hier darauf zurückführen, dass einzelne Chunks im Vergleich zum ersten Ansatz aktuell mehrfach aufgegriffen werden können, dadurch aus einem Chunk mehr als eine Anforderung generiert werden kann.

Eine große Schwachstelle des Ergebnisses vom zweiten Ansatz stellt die Vollständigkeit der Anforderungen dar. Hier liegt nahe, dass diese Schwäche auf die Qualität des Fragenschemas zurückzuführen ist (vgl. Anhang 03). Den Punkten, die im Fragenschema abgearbeitet werden, werden häufig dieselben Chunks zur Beantwortung zugeordnet während andere Chunks gar nicht betrachtet werden. Für die Optimierung des Fragenkatalogs liegen zwei Ansätze nahe: Es erfordert entweder ein stark auf den jeweiligen Input zugeschnittenes Fragenschema oder die Entwicklung eines möglichst allgemeinen Schemas für unterschiedliche Pflichtenhefte. Ein spezifisch zugeschnittenes Fragenschema würde jedoch die Übertragbarkeit des Lösungsansatzes auf andere Angebote/Pflichtenhefte einschränken, während ein universell optimales Schema nur schwer zu realisieren ist. Das Fragenschema stellt bezüglich der Vollständigkeit zumindest ein erhöhtes Risiko für Schwächen dar.

Die unzureichende Eindeutigkeit beim zweiten Ansatz kann in Teilen auf das Prompt Engineering zurückgeführt werden. Hier wird explizit vorgegeben, dass die Sätze z.B. mit „Das System...“ starten sollen. Durch Änderungen im Prompt Engineering muss diese Formulierungsvorgabe flexibler gestaltet werden und nicht aus allgemeinen Formulierungen, sondern näher beschreibenden Begriffen, welche direkt im inhaltlichen Kontext stehen, bestehen. Dadurch soll ermöglicht werden, dass z.B. eine Anforderung anstatt mit „Das System...“ mit „Ebene 0 aus Compartment 1...“ eingeleitet wird.

Die fälschlicherweise erkannten Anforderungen aus Kapitel 4.1.6 und 4.1.7 des Pflichtenheftes, sowie die Redundanzfreiheit sind analog zum ersten Ansatz zu betrachten.

Bezüglich der Rückverfolgbarkeit kann explizit für den zweiten Ansatz der k-Wert gesenkt und somit weniger Chunks pro Anforderung betrachtet werden. Dies scheint besonders empfehlenswert für simple und klar strukturierte Angebote, wie auch das in dieser Arbeit zugrundeliegende Angebot eines darstellt. Bei komplexeren Angeboten kann es vorteilhaft sein, weiterhin sehr viele Chunks für die Erstellung einer Anforderung zu betrachten, dies gilt es auf den konkreten Anwendungsfall anzupassen und exemplarisch zu überprüfen.

Auch die Limitationen des zweiten Ansatzes lassen sich potenziell zu großem Anteil auf verbesserungswürdige Umsetzung des erstellten Ansatzes zurückführen. Dadurch ist ein struktureller Vergleich der beiden Ansätze erschwert. Einzelne aussagekräftige Erkenntnisse können trotzdem gewonnen werden. Diese werden im folgenden Kapitel dargelegt.

7 Fazit & Ausblick (Luca Russek)

Grundsätzlich ist festzuhalten, dass beide Ansätze keine vollständig fehlerfreien Anforderungslisten generieren. Es liegt nahe, dass ein großer Teil der identifizierten Schwächen mehr auf die konkrete Umsetzung als auf die zugrundeliegende Struktur der Ansätze zurückzuführen ist. Dadurch ist der Vergleich auf Basis dieser Schwächen nur eingeschränkt aussagekräftig. Für weiterführende Untersuchungen sollte daher spezifiziert und validiert werden, welche Schwächen, durch die in Kapitel 6 beschriebenen Optimierungsansätze behoben werden können, um tatsächliche Limitationen der Ansätze klarer herausarbeiten zu können.

Aus den Ergebnissen geht jedoch hervor, dass die Qualität des Fragenschemas beim RAG-basierten Ansatz entscheidend für die Vollständigkeit der generierten Anforderungsliste ist. Dadurch entsteht eine potenzielle Schwachstelle, die beim ersten Ansatz in dieser Form nicht besteht. Wie bereits in Kapitel 5.1 dargelegt, wurden mehrere Abschnitte im Output nicht berücksichtigt, da die entsprechenden Chunks für keine Frage des Fragenschemas als relevant erachtet wurden.

Beim doppelstufigen LLM-Ansatz hingegen wird jeder Chunk verarbeitet und dadurch zumindest jedes Kapitel in der Anforderungsliste „repräsentiert“. Auch hier besteht weiterer Forschungsbedarf, insbesondere hinsichtlich der Frage, inwieweit ein detaillierteres Fragenschema die Vollständigkeit beim RAG-Ansatz verbessern kann.

Grundsätzlich lässt sich jedoch festhalten, dass der doppelstufige LLM-Ansatz im Hinblick auf die Vollständigkeit eine strukturelle Schwachstelle weniger aufweist, da alle Chunks sicher berücksichtigt werden. Für eine abschließende Bewertung der Eignung des Ansatzes ist jedoch weiter zu untersuchen, inwieweit die beobachteten Schwächen unter Umständen auf die Methodik selbst und nicht primär auf deren Umsetzung zurückzuführen sind. Speziell für komplexe Angebote, bei denen die Betrachtung des Kontexts aus z.B. drei vorhergehende Chunks und dem eigentlich betrachteten Chunk, lässt sich die Vermutung anstellen, dass hier strukturell nicht ausreichend Kontext einfaktoriert wird, um vollständige und eindeutige Anforderungen zu formulieren.

Auch die Rückverfolgbarkeit ist bei ersten Ansatz besser, solange der k-Wert bei zweiten Ansatz nicht auf 1 gestellt wird. Sobald bei zweiten Ansatz mehrere Chunks einbezogen werden, ist nicht mehr klar rückzuverfolgen, welcher Chunk tatsächlich welchen Einfluss auf den Inhalt der tatsächlichen Anforderung hat. Gleichzeitig gilt auch hier: Für komplexere oder schlechter strukturierte Angebote kann es unabdingbar sein, mehrere Chunks zu betrachten, während bei simplen und gut strukturierten Angeboten die simplere Lösung in Form des ersten Ansatzes ausreichend ist.

Zusammenfassend lässt sich somit festhalten, dass der doppelstufige LLM-Ansatz sowohl hinsichtlich der Rückverfolgbarkeit eindeutiger, oder zumindest gleichwertig bei k=1 für den zweiten Ansatz ist, als auch hinsichtlich der Vollständigkeit, aufgrund der garantierten Berücksichtigung aller Chunks, eine potenzielle Fehlerquelle eliminiert. Gelingt es nach weiterer Forschung bei diesem Ansatz, die identifizierten Schwächen durch Optimierungen in der Umsetzung weiter zu reduzieren, kann dieser durchaus eine valide und insgesamt stabilere Methode zur Generierung einer Anforderungsliste aus einem Angebot darstellen. Gleichzeitig stellt der zweite Ansatz in seiner Struktur einen Ansatz dar, der vor allem für komplexe, unsortierte Angebote das Potenzial für eine robustere Anwendung und bessere Kontexteinbeziehung liefert. Auch hier gilt es weitere Nachforschungen anzustellen. Es muss herausgefunden werden, ob die Limitationen vom ersten Ansatz tatsächlich auf ausbaufähige Umsetzung zurückzuführen sind und gleichzeitig überprüft werden, inwieweit dieser Ansatz auch für weniger strukturierte Angebote anwendbar ist.

Quellenverzeichnis

Alkhamis, M. A., 2023. *Extraktion textueller Informationen aus.* [Online]
Available at: https://eprints.dbis.informatik.uni-rostock.de/1096/1/15.03.23.Mahmoud.Khamis.pdf
[Zugriff am 26. März 2026].

Almeida, F. & Xexéo, G., 2023. Word Embeddings: A Survey. p. 11.

Amiri, M. & Bocklitz, T., 2025. Chunk Twice, Embed Once: A Systematic Study of Segmentation and Representation Trade-offs in Chemistry-Aware Retrieval-Augmented Generation. p. 46.

Bai, J. et al., 2023. *Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond.* [Online]
Available at: https://arxiv.org/pdf/2308.12966
[Zugriff am 26. März 2026].

Brown, A., Roman, M. & Devereux, B., 2025. *A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges.* [Online].

Chase, H., 2026. *Github.* [Online]
Available at: https://github.com/langchain-ai/langchain
[Zugriff am 20 03 2026].

Chen, X. et al., 2015. An Empirical Study of OpenAI API Discussions on Stack Overflow. *IEEE Transactions on Software Engineering,* 14(8).

Dong, Z. et al., 2021. *Transformation from human-readable documents and archives in arc welding domain to machine-interpretable data.* [Online]
Available at: https://pdf.sciencedirectassets.com/271439/1-s2.0-S0166361521X00048/1-s2.0-S0166361521000464/main.pdf?X-Amz-Security-Token=IQoJb3JpZ2luX2VjEPj%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLWVhc3QtMSJIMEYCIQDKNi3ShfM8wN65BQFAAwhpg0o95xMWGzJNkuWn7XaUuwIhAOrSgoSRe2Ys
[Zugriff am 26. März 2026].

Ehrlenspiel, K., Kiewert, A., Lindemann, U. & Mörtl, M., 2020. *Kostengünstig Entwickeln und Konstruieren.* 8. Hrsg. s.l.:Springer.

Farnschläder, T., 2024. *Tqdm Python: A Guide With Practical Examples.* [Online]
Available at: https://www.datacamp.com/tutorial/tqdm-python
[Zugriff am 28. März 2026].

Gazoni, E. & Clark, C., 2024. *openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files.* [Online]
Available at: https://openpyxl.readthedocs.io/en/stable/
[Zugriff am 28. März 2026].

GWDG, 2026. *SAIA Scalable AI Accelerator.* [Online]
Available at: https://docs.hpc.gwdg.de/services/ai-services/saia/index.html
[Zugriff am 28 März 2026].

Hoque, M., 2024. *Medium.* [Online]
Available at: https://medium.com/@asmmorshedulhoque/demystifying-word-embeddings-count-based-vs-model-based-techniques-394d3834a66d
[Zugriff am 22 03 2026].

INCOSE Publications Office, 2023. *INCOSE Guide to Writing Requirements V4 – Summary Sheet.* [Online]
Available at: https://www.incose.org/wp-content/uploads/legacy/working-groups/requirements-wg/guidetowritingrequirements/incose_rwg_gtwr_v4_summary_sheet.pdf
[Zugriff am 26. März 2026].

International Organization for Standardization, I. E. C. I. o. E. a. E. E., 2018. *Systems and software engineering - Life cycle processes - Requirements engineering.* Schweiz: International Organization for Standardization, Institute of Electrical and Electronics Engineers.

Klesel, M. & Wittman, H. F., 2025. Retrieval-Augmented Generation (RAG). *Business & Information Systems Engineering,* I(67), pp. 551-561.

Krippendorff, K., 2011. *Computing Krippendorff's alpha-reliability,* State College, Pennsylvania: Pennsylvania State University.

Kshirsagar, A., 2024. *Enhancing RAG Performance Through Chunking and Text Splitting Techniques.* [Online]
Available at:
https://www.researchgate.net/publication/383998204_Enhancing_RAG_Performance_Through_Chunking_and_Text_Splitting_Techniques
[Zugriff am 26. März 2026].

Li, L. et al., 2025. *Exploring the Impact of Temperature on Large Language Models: Hot or Cold?.* [Online]
Available at: https://arxiv.org/pdf/2506.07295
[Zugriff am 26. März 2026].

Lindemann, U., 2009. *Methodische Entwicklung technischer Produkte.* 3. Hrsg. s.l.:Springer.

Li, Z. et al., 2025. Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence,* I(8).

McKinnney, W., 2010. *Data Structures for Statistical Computing in Python.* [Online]
Available at: https://proceedings.scipy.org/articles/Majora-92bf1922-00a.pdf
[Zugriff am 28. März 2026].

meta-llama, 2024. *Llama-3.3-70B-Instruct.* [Online]
Available at: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
[Zugriff am 22. März 2026].

Minh, N. N. et al., 2025. *TURNING UP THE HEAT: MIN-p SAMPLING FOR.* [Online]
Available at: https://arxiv.org/pdf/2407.01082
[Zugriff am 26. März 2026].

Neha, F., Bhati, D. & Shukla, D. K., 2025. Retrieval-Augmented Generation (RAG) in Healthcare: A Comprehensive Review. *MDPI AI,* 6(9).

Park, C. R., Heo, H., Suh, C. H. & Shim, W. H., 2025. Uncover This Tech Term: Application Programming Interface for Large Language Models. *Korean Journal of Radiology,* 26(8), pp. 793-796.

Sangeethasaravanan, 2025. *Unlocking PDF Data the Smart Way: A Practical Guide to pdfplumber.* [Online]
Available at: https://sangeethasaravanan.medium.com/unlocking-pdf-data-the-smart-way-a-practical-guide-to-pdfplumber-3c80b5d9d491
[Zugriff am 22. März 2026].

Schiller, K. A., Haddad, M.-S. & Seibel, A., 2025. ReqGPT: a fine-tuned large language model for generating requirements documents. *Proceedings of the Design Society,* Issue 5, p. 10.

Schwaber-Cohen, R. & Patel, A., 2025. *Pinecone.* [Online]
Available at: https://www.pinecone.io/learn/chunking-strategies/

spaCy (a), kein Datum *Library Architecture.* [Online]
Available at: https://spacy.io/api
[Zugriff am 22. März 2026].

spaCy (b), kein Datum *Training Pipelines & Models.* [Online]
Available at: https://spacy.io/usage/training
[Zugriff am 26. März 2026].

Thevapalan, A., 2025. *A Beginner's Guide to The OpenAI API: Praktisches Tutorial und Best Practices.* [Online]
Available at: https://www.datacamp.com/de/tutorial/guide-to-openai-api-on-tutorial-best-practices
[Zugriff am 28. März 2026].

Tuychlev, B., 2025. *Python Umgebungsvariablen: Was du wissen musst.* [Online]
Available at: https://www.datacamp.com/de/tutorial/python-environment-variables
[Zugriff am 28. März 2026].

VAGO Solutions, 2024. *VAGO solutions Llama-3.1-SauerkrautLM-70b-Instruct.* [Online]
Available at: https://huggingface.co/VAGOsolutions/Llama-3.1-SauerkrautLM-70b-Instruct
[Zugriff am 22. März 2026].

Verma, A., 2023. *Understanding temperature, top_p, top_k, logit_bias in LLM parameters.* [Online]
Available at: https://aviralrma.medium.com/understanding-llm-parameters-c2db4b07f0ee [Zugriff am 26. März 2026].

Zhao, P. et al., 2026. Retrieval-Augmented Generation for AI-Generated Content: A Survey. *Data Science and Engineering,* Issue 11, pp. 1-29.

Zowghi, D. & Gervasi, V., 2002. *The Three Cs of Requirements : Consistency , Completeness , and Correctness,* s.l.: s.n.

Anhangsverzeichnis

Anhang 1: V2_Pflichtenheft_komprimiert.pdf XIII
Anhang 2: Flussdiagramm - Pipeline - Doppelstufiger LLM Ansatz XXIV
Anhang 3: Fragenkatalog - RAG System - Skript XXV
Anhang 4: Flussdiagramm - Pipeline - RAG System XXVI
Anhang 5: ZIP-Ordnerverzeichnis „01_Finale Abgabe - KI Projekt“ und übermittelter GitHub Link XXVII

Anhang 1: V2_Pflichtenheft_komprimiert.pdf

Dieses Dokument fungierte als primärer Trainingsdatensatz für beide eingesetzten LLM-Modelle. Entsprechend den Ausführungen in Kapitel 2.1 wurde aus dem zur Verfügung gestellten Literatur- und Materialangebot eine systematische Eingrenzung der relevanten Kapitel vorgenommen. Für die methodische Übertragbarkeit und Skalierbarkeit des Systems ist die erfolgreiche Verarbeitung heterogener Datenformate – wie Tabellen, technische Skizzen, Konzeptzeichnungen und komplexe Bilddaten – ein entscheidender Faktor.

4 Angebotsumfang

Aus dem im vorangegangenen Abschnitt beschriebenen Lösungskonzept entstehen für den Auftragnehmer die im Folgenden aufgeführten Lieferumfänge.

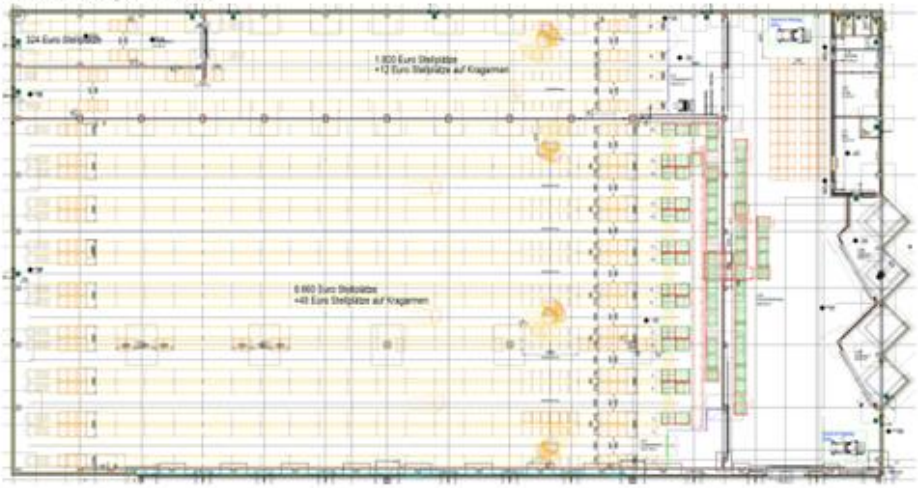
Keines der im Lieferumfang enthaltenen Gewerke (z.B. Regal, Bühnen etc.) berücksichtigt Schnittstellen mit anderen Gewerken (z.B. Sprinkler, Technische Gebäudeausrüstung, Elektroinstallationen etc.) sofern nicht explizit anders beschrieben. Das betrifft z.B. zusätzliche Lasten, Befestigungspunkte, Freiräume innerhalb der Konstruktion, Gesamtprojektterminplan und Ähnliches.

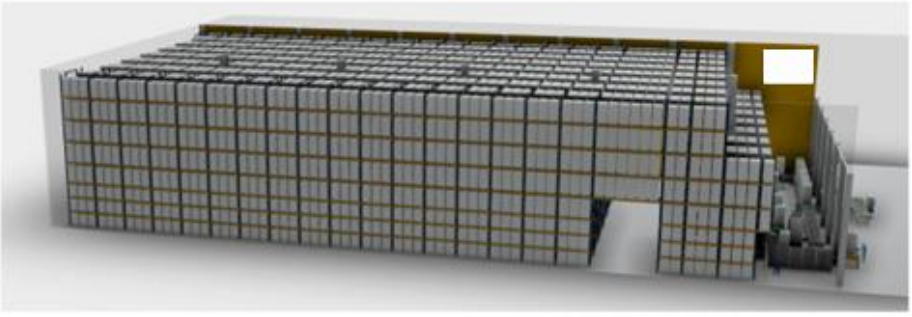
4.1 Paletten-Regalanlage / Sprinkleranlage

4.1.1 Regalanlage

4.1.1.1 Beschreibung des Regalaufbaus

Die Dimensionen und Bestandteile des Regals sowie die Anzahl an Stellplätzen im Regal ist dem angehängten Layout zu entnehmen. Weitere Details sind nicht zwingend im Layout dargestellt, jedoch im Lieferumfang enthalten.





35 / 132

4.1.1.1 Kapazität

- ca. 10.844 Euro-Palettenplätze*

*) finale Lagerplatzkapazität abhängig vom Fluchtwegs Konzept

4.1.1.2 Höhenaufteilung

Compartment 1

Anzahl Paletten übereinander: 8 + Bodenebene

Ebene	Oberkante Regalauflage	Lichte Höhe
0		1110 mm
1	1.250mm	1110 mm
2	2.500 mm	1110 mm
3	3.750 mm	1.410 mm
4	5.300 mm	1.760 mm
5	7.200 mm	1.760 mm
6	9.100 mm	1.960 mm
7	11.200 mm	1.960 mm
8	13.300 mm	Top mm (2.060mm)

Compartment 2 / 3

Anzahl Paletten übereinander: 8 + Bodenebene

Ebene	Oberkante Regalauflage	Lichte Höhe
0		1110 mm
1	1.250mm	1110 mm
2	2.500 mm	1110 mm
3	3.750 mm	1.410 mm

4	5.300 mm	1.760 mm
5	7.200 mm	1.760 mm
6	9.100 mm	1.760 mm
7	11.200 mm	1.760 mm
8	12.900 mm	Top mm (2.435mm)

4.1.1.3 Regalauflagen

Auflagenlänge 1:	2.950 mm
Auflagentyp:	AUF'B'E0444 – 140/50/1.5 S235
Fachlast:	3.000 kg pro Auflagenpaar

Auflagenlänge 2:	2.000 mm
Auflagentyp	AUF'B'E414 – 140/50/1.5 S355MC
Fachlast:	2000 kg pro Auflagenpaar

Auflagenlänge 3:	1.300 mm
Auflagentyp	AUF'B'E414 – 140/50/1.5 S355MC
Fachlast:	1.000 kg pro Auflagenpaar

P&D Auflage:	1.050mm
Auflagentyp:	AUF'B'E0435/15 – 145/50/1.5 S355MC

Durchfahrt:	5.900 mm
Auflagentyp	IPE Profil – 180/90/1.5 S235
Fachlast:	3000 kg pro Auflagenpaar

4.1.1.4 Regalständer

Höhe Ständerrahmen:	13500 mm
Höhe Endständer:	13500 mm
Ständertiefe:	1.100 mm
Ständertyp:	STD'B'28P (Z2-config)
Profilbreite:	120/65/2.5 S420MC
Maximale Feldlast:	23.147 kg – 94 % gleichm. verteilte Last
Effektive Pressfläche der Fußplatte:	199,4 cm²
Maximale Flächenpressung:	65,37 kg/cm²

Für die angegebene max. Feldlast sind die angegebenen Etagenhöhen maßgeblich.

Diagonalverbände:		Ja	X	Nein

4.1.2 Berechnungsgrundlage

Norm: EN 15512:2020+A1:2022

Das Angebot ist unter Berücksichtigung der DIN EN 15512:2021 erstellt.

Regalklasse: 300B

Max. Durchbiegung der Auflage: <10mm

D < 10 mm

Die genannten Belastungswerte verstehen sich bei gleichmäßig verteilter Last.
Einflüße von Erdbebenlasten nach EN 16681 wurden in der Berechnung berücksichtigt.

		Ja	X	Nein

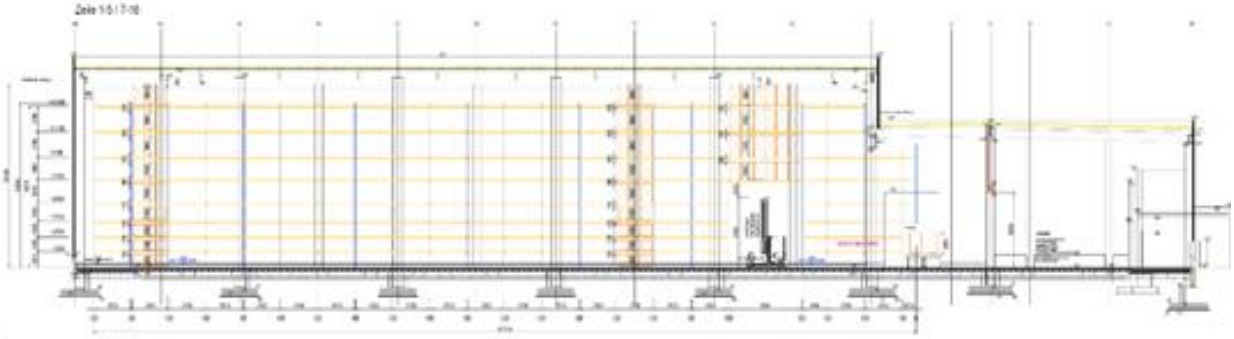
4.1.3 Oberflächenbehandlung

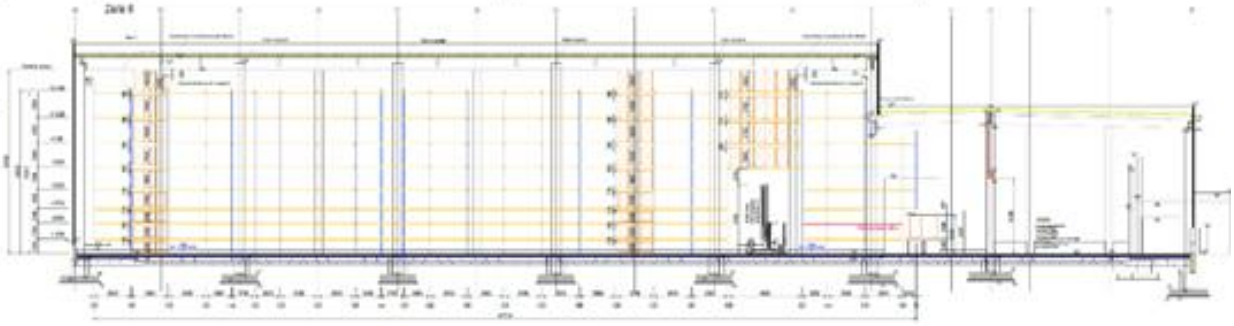
Fast alle Systembauteile werden mit Epoxidharz-Polyesterpulverlack lackiert und nachfolgend im Polymerisationsofen eingebrannt. Dabei kommen folgende Standardfarbtöne zum Einsatz:

für die Ständer:	RAL 5010 (Enzianblau)/Fachwerk galvanisiert
für die Auflagen:	RAL 1007 (Narzissengelb)
für den Anfahrschutz:	RAL 1004 (Goldgelb)
für diverse andere Bauteile:	galvanisiert

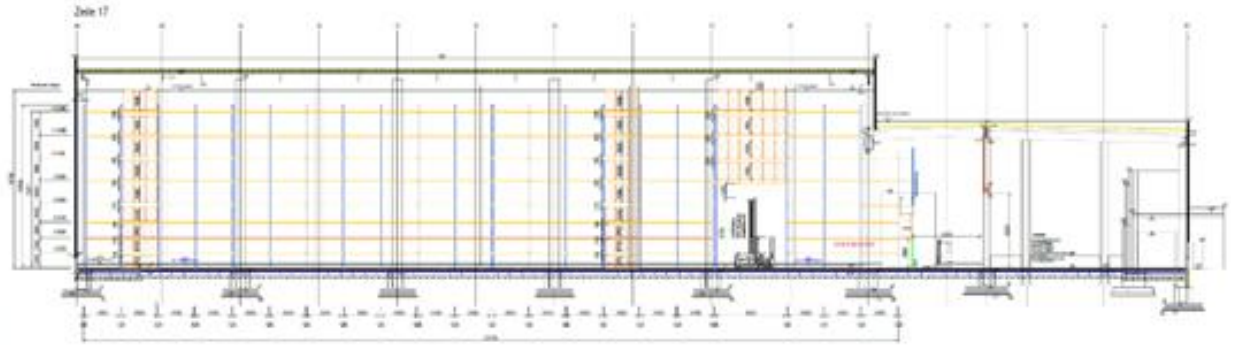
4.1.4 Schnittansichten

Compartment 1 (9 Ebenen)



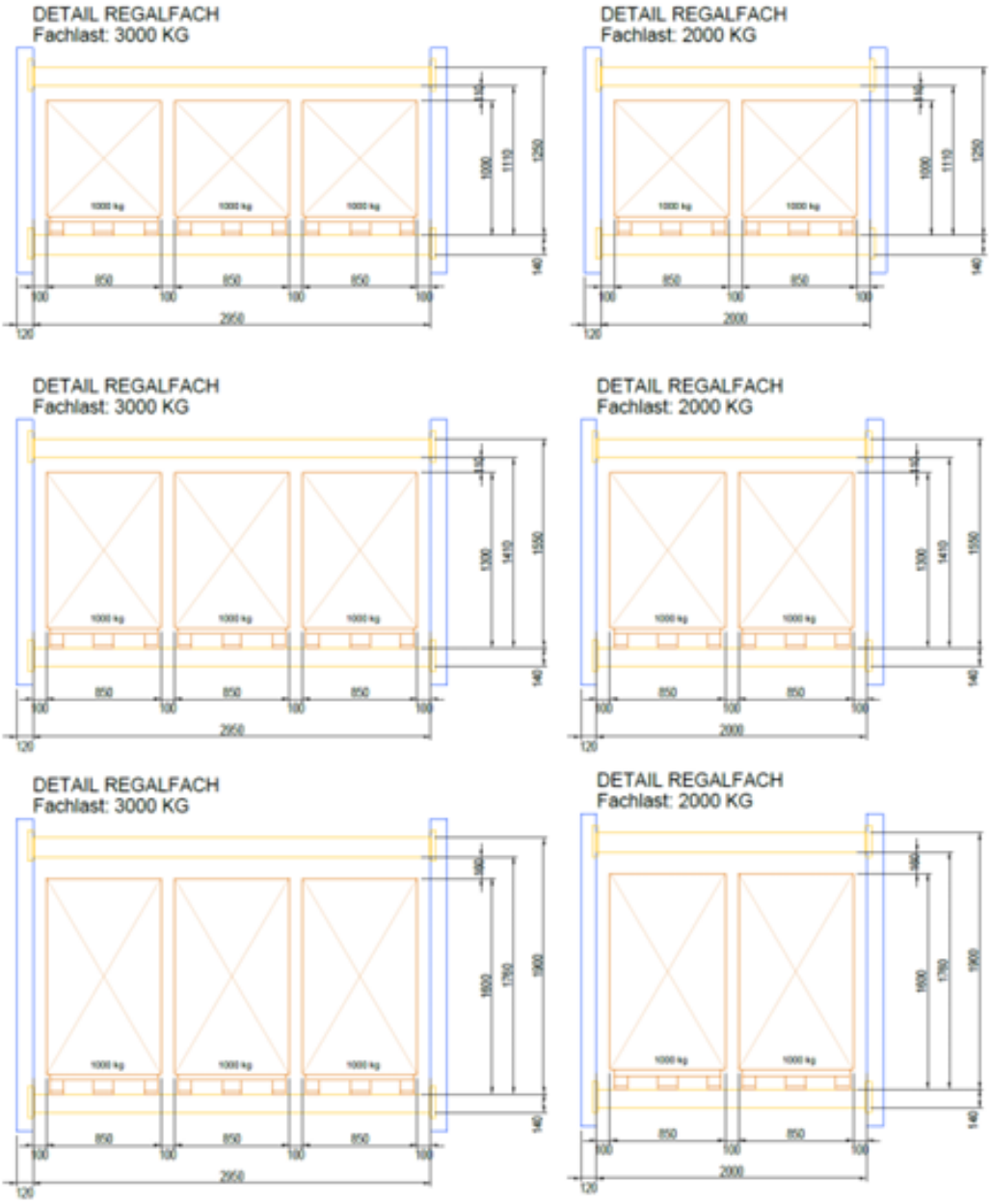


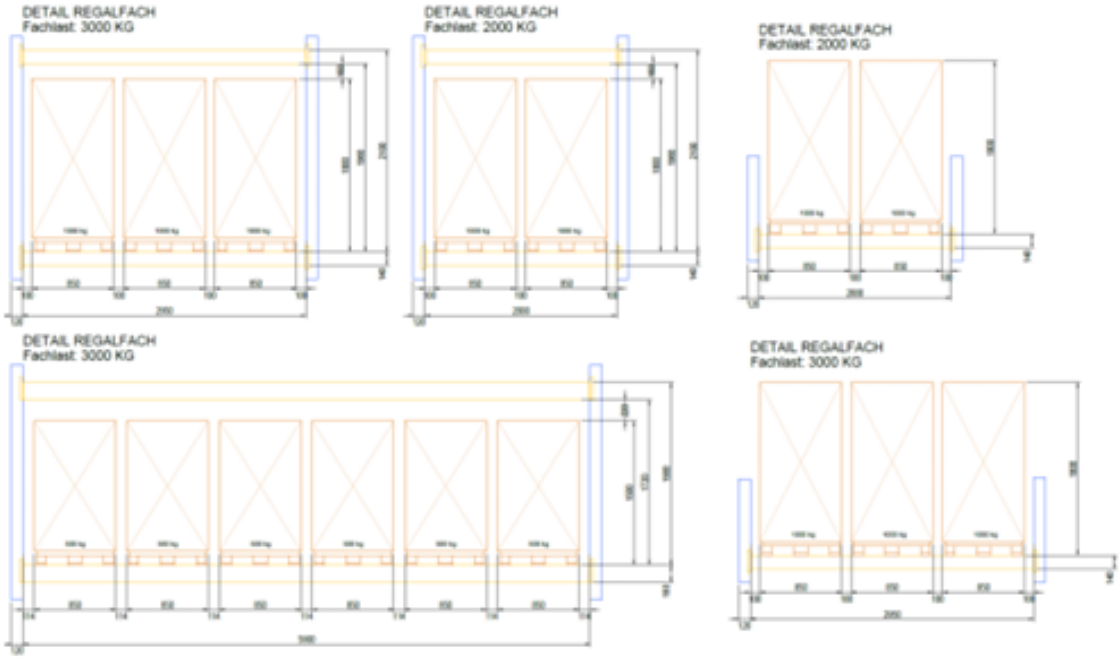
Compartment 2 / 3 (9 Ebenen)



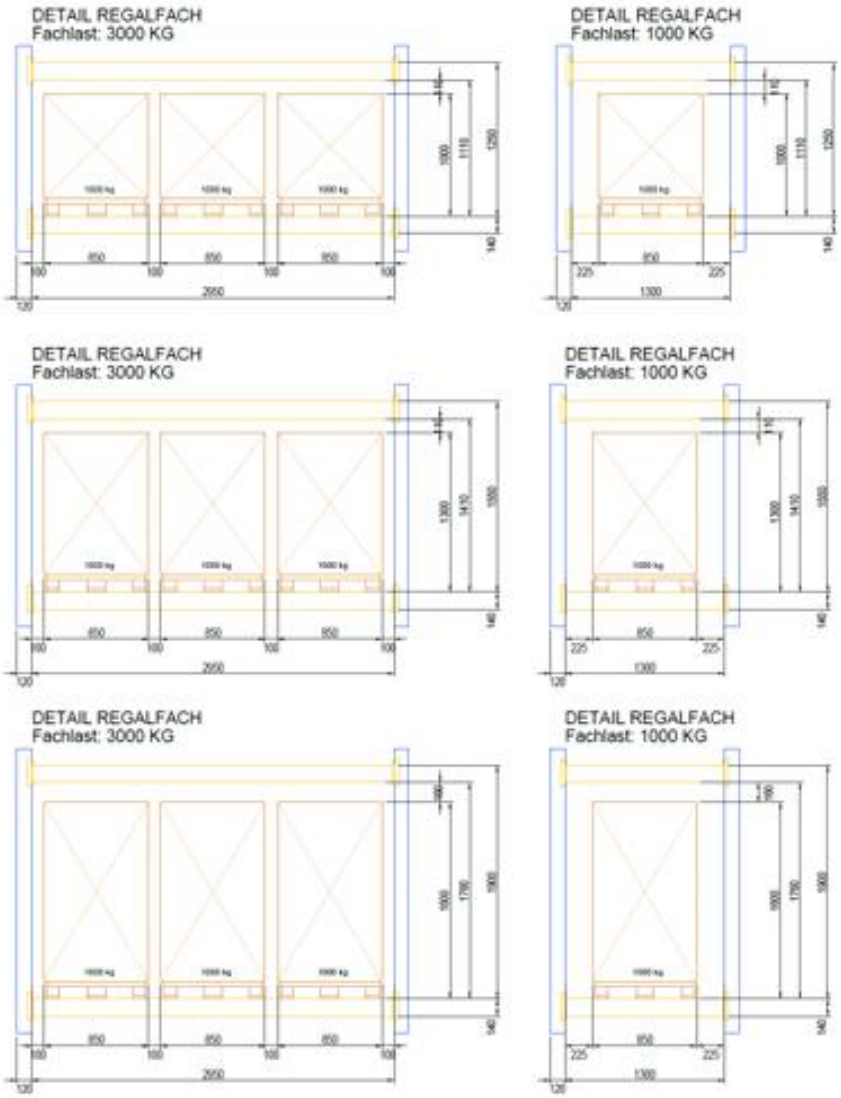
4.1.5 Regalfächer Detail

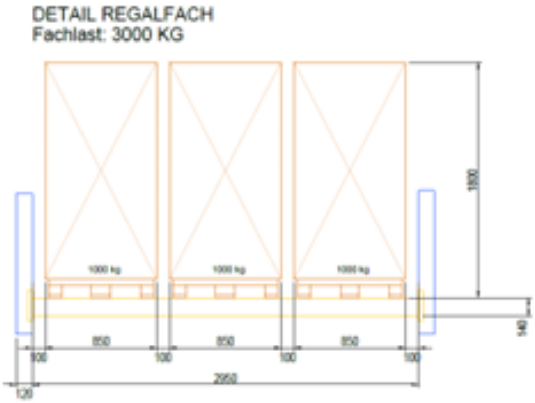
Compartment 1

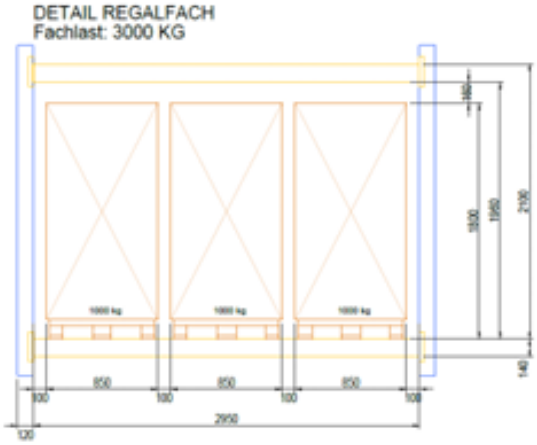



DETAIL REGALFACH
Fachlast: 3000 KG
DETAIL REGALFACH
Fachlast: 2000 KG
DETAIL REGALFACH
Fachlast: 2000 KG
DETAIL REGALFACH
Fachlast: 3000 KG
DETAIL REGALFACH
Fachlast: 3000 KG

Compartment 2 / 3




DETAIL REGALFACH
Fachlast: 3000 KG
1000 kg
1000 kg
1000 kg
1800
140
850
850
850
100
100
100
100
2950
120


DETAIL REGALFACH
Fachlast: 3000 KG
1000 kg
1000 kg
1000 kg
180
1800
1950
2130
140
850
850
850
100
100
100
100
2950
120

DETAIL REGALFACH
Fachlast: 3000 KG
500 kg
500 kg
500 kg
500 kg
500 kg
500 kg
299
1500
1725
1900
180
850
850
850
850
850
850
114
114
114
114
114
114
5900
120

4.1.6 Lieferkonditionen Regalanlage

Normen	Die Ausführung des gelieferten Produkts entspricht den relevanten Vorschriften und Richtlinien. Nähere Informationen hierzu finden Sie im Kapitel „Technische Informationen“ im Abschnitt „Berechnungsgrundlage“.
Zahlung	Gemäß Vertrag
Lieferung	Frachtfrei
Montage	Inklusive
Staplergestellung	
Abladen	
Verbringen zum Montageort	

- Eine WHG-Verdübelung mit entsprechender Dokumentation ist nicht in dem Angebot inkludiert.

4.1.7 Montagebedingungen Regalanlage

Wir gehen bei unserer Kalkulation von einer besenreinen Halle aus, in der ungehindert gearbeitet werden kann.
Für den Hallenboden wird eine Mindestbetongüte C20/25 mit entsprechender Bewehrung gemäß DIN EN 206-1 / DIN 1045-2 vorausgesetzt. Der Hallenboden muss mindestens 20 cm dick sein und eine Bodenverankerung mit Spreizdübeln zulassen; Bohrtiefe ca. 15 cm. Nachweise über Tragfähigkeit und zulässige Flächenpressung des Fußbodens hat der Auftraggeber zu erbringen.

- Aufgrund einer bauseitigen Fußbodenheizung wird eine regalseitige Verdübelung von 10cm vorgesehen

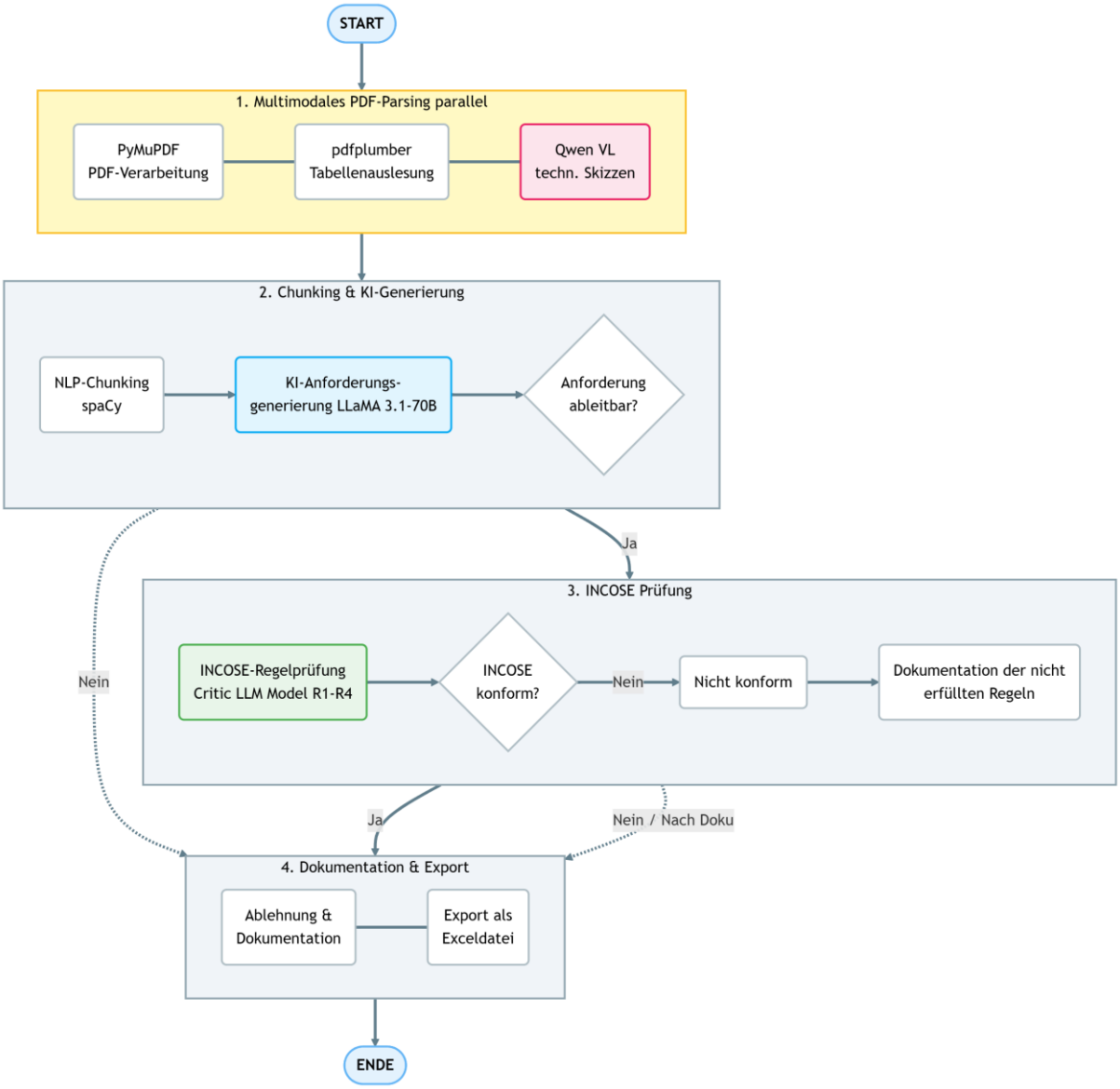
Der Fußboden darf nicht aus einem Material bestehen, das zu chemischen Reaktionen mit dem Regal und der Verankerung führt (z.B.: magnesithaltig).
Alle baurechtlichen Vorschriften sind vom Auftraggeber der Regalanlage mit der zuständigen Baubehörde abzuklären.
Besondere Belastungssituationen sind Einwirkungen auf Regalanlagen, die bei der Berechnung der Regalanlage von entsprechender Bedeutung sind und welche nicht mit den Standardregelwerken (z.B. bei Palettenregalen EN 15512 bzw. bei Kragarmregalen EN1993) abgedeckt sind.
Besondere Belastungssituationen sind unter anderen Umwelteinflüsse wie, Einwirkungen aus Erdbeben, Wind & Schnee (Aufstellung im Freien). Standardmäßig werden derartige besondere Belastungssituationen bei der Konstruktion von Regalanlagen nicht berücksichtigt.
Die Prüfung, ob aufgrund der betrieblichen Umgebung, sonstiger lokaler Gegebenheiten und den anzuwendenden gesetzlichen Vorschriften/ technischen Normen besondere Belastungssituationen zu berücksichtigen sind, obliegt dem Auftraggeber. Dies gilt im besonderen Maß für die Berücksichtigung von Belastungen, die sich daraus ergeben, dass das Regal in erdbebengefährdeten Gebieten zur Aufstellung kommen soll. Für verstellbare Palettenregale stellt beispielsweise die europäische technische Norm EN 16681 die Leitsätze für erdbebensichere Bemessung auf. Der Auftraggeber hat besondere Belastungssituationen spätestens zur Auftragserteilung schriftlich mitzuteilen, damit diese bei dem Auftrag berücksichtigt werden können.
Veränderungen am Regal bedürfen der Zustimmung des Herstellers, dieses gilt insbesondere auch für Umbauten, die abweichende Etagenaufteilungen gemäß den vorhandenen Belastungsschildern vorsehen.

Andernfalls entfallen die Gewähr und jegliche Verantwortung des Herstellers für die Herstellung und statische Auslegung des Regals.
Bei Umbauten ist die Aufbauanleitung zu beachten.
Ein prüffähiger statischer Nachweis ist in unserem Lieferumfang nicht enthalten, dieser kann bei Bedarf gegen Gebühr angefordert werden.

Schmalganglager
Der tragende Untergrund ist nach DIN 18202 auszulegen.
Insbesondere ist die DIN 15185, Teil 1, als Mindestanforderung zu beachten, wobei jedoch der Fahrboden innerhalb der Regalgassen nach der VDMA-Richtlinie „Böden für den Einsatz von Schmalgang-Flurförderzeugen“ (www.vdma.org) auszulegen ist.
Der Erdableitungswiderstand RE gemessen nach DIN 51953 darf 106 Ohm nicht überschreiten.
Hinsichtlich der Regalanlage ist die EN 15620 zu beachten.
Für weitere Hinweise siehe Broschüre „Das Schmalganglager“.

Anhang 2: Flussdiagramm - Pipeline - Doppelstufiger LLM Ansatz

Um die methodische Transparenz des ersten Ansatzes (zweistufige LLM-Methode) zu erhöhen, wird dessen Architektur durch ein Flussdiagramm grafisch aufbereitet. Die Darstellung skizziert detailliert die lineare Pipeline der Textverarbeitungsstufen und die daraus resultierenden Abhängigkeiten. Dies flankiert die Erläuterung des implementierten Python-Skripts, das innerhalb der beigefügten Ordnerstruktur referenziert wird.



Anhang 3: Fragenkatalog - RAG System - Skript

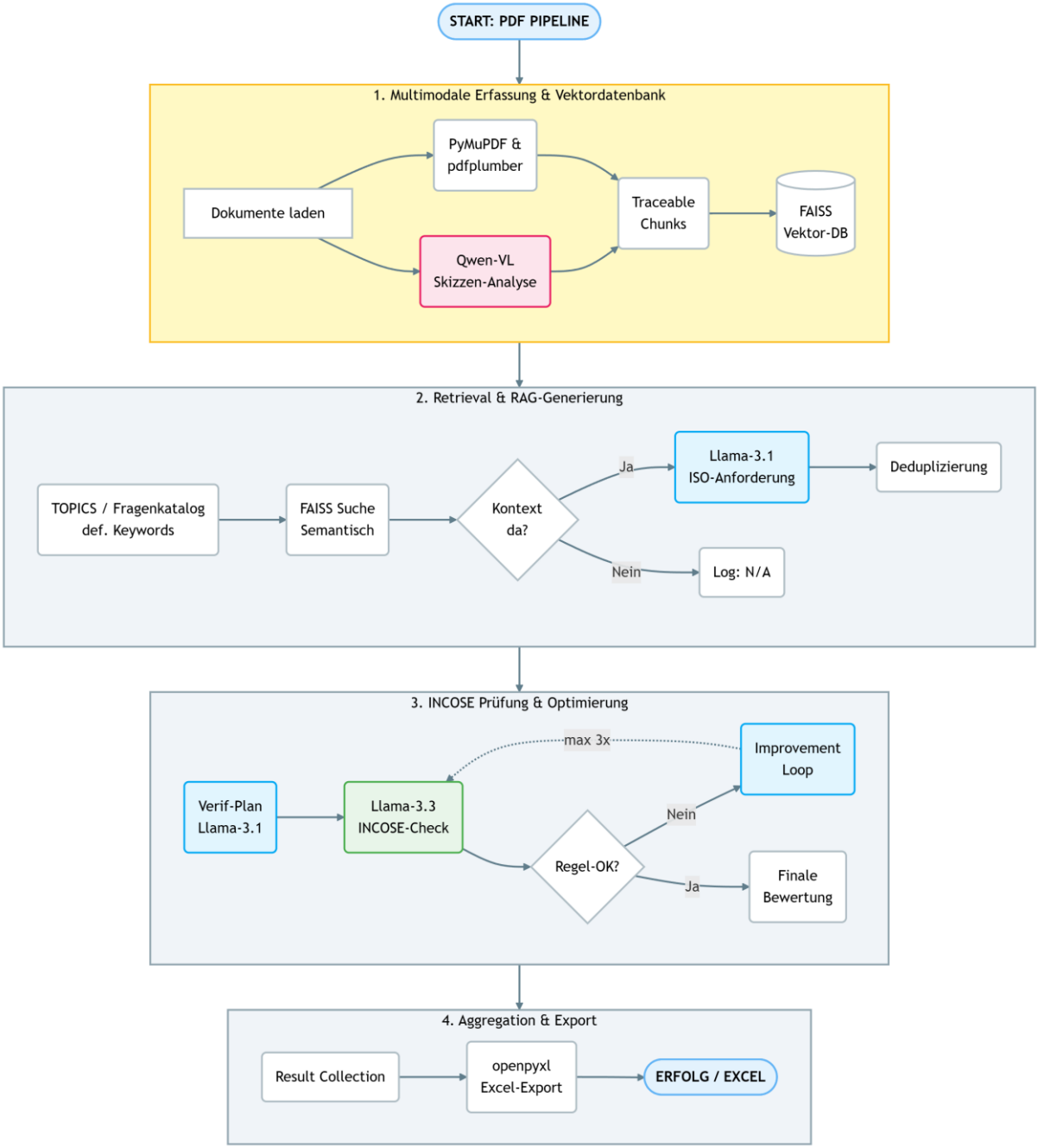
Der nachfolgend dargestellte Fragenkatalog dient im Rahmen des zweiten Ansatzes, dem Python-Skript mit integrierter RAG-Pipeline, der gezielten Steuerung des Retrievers. Zur Sicherstellung einer hochauflösenden und detaillierten Textverarbeitung wurde dieses Dokument, entsprechend den Ausführungen in Kapitel 4.3, systematisch auf das komprimierte Pflichtenheft abgestimmt und im Skript eingefügt.

Anforderungsthemen:

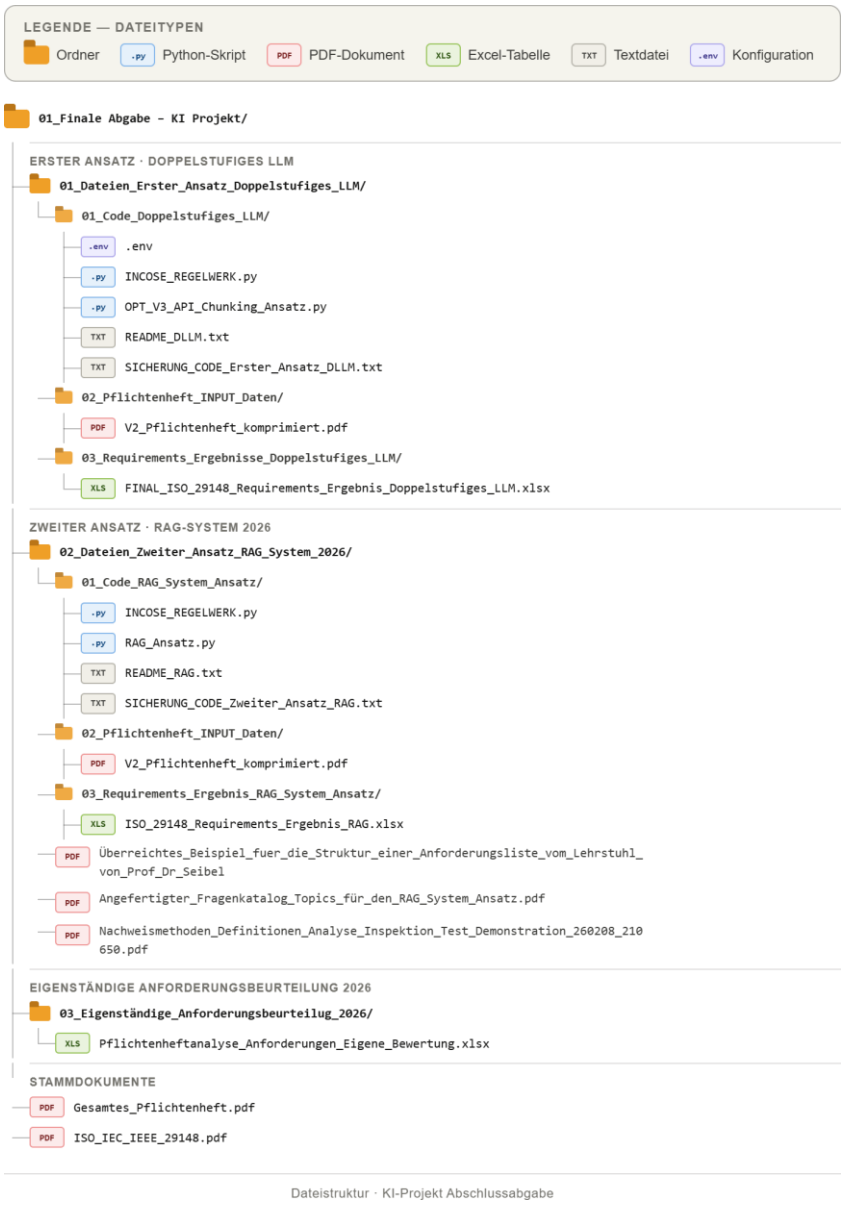
HAUPTMERKMAL	*BEISPIEL*
GEOMETRIE	Abmessungen Ladehilfsmittel max. Ladungsgrößen max. Höhe der Transporteinheiten Gassen-/AST-Maße
KINEMATIK	Stabilität der Ladegüter Max. Fahrgeschwindigkeit Anforderungen an Bewegungsabläufe
KRÄFTE	max. Gewicht der Transporteinheiten Resttragfähigkeit des Fahrzeugs Durchbiegung
ENERGIE	Batteriespannung Ersatzstromanlage Schutzarten
STOFF	Material und Produktspezifikationen
SIGNAL	Schnittstellen Scan und identifikationen
SICHERHEIT	Brandschutz Ausrüstung Lichtverhältnisse Notstrom
ERGONOMIE	Bediengeräte Wartungsbereiche
KONTROLLE	Qualifizierung und Normen Detektionseinheiten
MONTAGE	Aufbauspezifikationen Montageabwicklung
TRANSPORT	Transporteinheiten Transportdauer
GEBRAUCH	Betriebsbedingungen
INSTANDHALTUNG	Wartungsabwicklung Verschleißreduzierende Materialien
KOSTEN	Mehrkosten Budget

Anhang 4: Flussdiagramm - Pipeline - RAG System

Um die Architektur des implementierten RAG-Ansatzes methodisch transparent darzustellen, wird die textliche Beschreibung durch ein Flussdiagramm flankiert. Diese grafische Darstellung bildet die vollständige Pipeline des Python-Skripts ab und veranschaulicht detailliert die logischen Abhängigkeiten und den Datenfluss über alle Verarbeitungsstufen hinweg.



Anhang 5: ZIP-Ordnerverzeichnis „01_Finale Abgabe - KI Projekt“ und übermittelter GitHub Link

Sowohl bei der Darlegung der implementierten Programmieransätze als auch im Rahmen der Ergebnispräsentation wird im Haupttext explizit meist indirekt auf Anhang 5 referenziert. Dadurch werden die beigefügten digitalen Ressourcen, aus denen die dargelegten Informationen und Datensätze abgeleitet wurden, strukturell in die Arbeit eingebunden. Zur methodischen Transparenz und besseren Orientierung wird im Folgenden ein detailliertes Verzeichnis der angehängten digitalen Dokumente und Ordnerstrukturen aufgeführt. Diese Systematik ermöglicht einen effizienten Zugriff auf die umfangreichen Begleitdaten und stellt sicher, dass sämtliche im Haupttext erwähnten Dateien in der Anlage eindeutig lokalisiert werden können.

Link zu dem GitHub Ordner, zusätzlich zum übermittelten ZIP-Ordner:
https://github.com/oletrosin/01_Finale_Abgabe_KI_Projekt